

LTSF-Linear: A simple, strong baseline for Long-Term Forecasts

Nguyễn Quốc Thái
Nguyễn Phúc Thịnh

Hồ Quang Hiên
Đinh Quang Vinh

I. Giới thiệu

Dự báo chuỗi thời gian, đặc biệt là trong lĩnh vực tài chính, là một trong những bài toán kinh điển và cũng đầy thách thức của ngành AI. Việc thành công trong dự đoán giá cổ phiếu không chỉ đem về cho ta lợi nhuận, mà còn là một công cụ cốt lõi để quản trị rủi ro. Tuy nhiên, thử thách của bài toán này thực sự không nằm ở việc đoán giá của ngày mai, mà là việc “nhìn xa” (Long-term Time Series Forecasting - LTSF). Dữ liệu tài chính nổi tiếng là cực kỳ “nhiều” (noisy), phi tuyến tính (non-linear) và không có tính dừng (non-stationary). Dự đoán vài ngày tới đã khó, dự đoán hàng tuần hay hàng tháng là một bài toán ở cấp độ hoàn toàn khác.



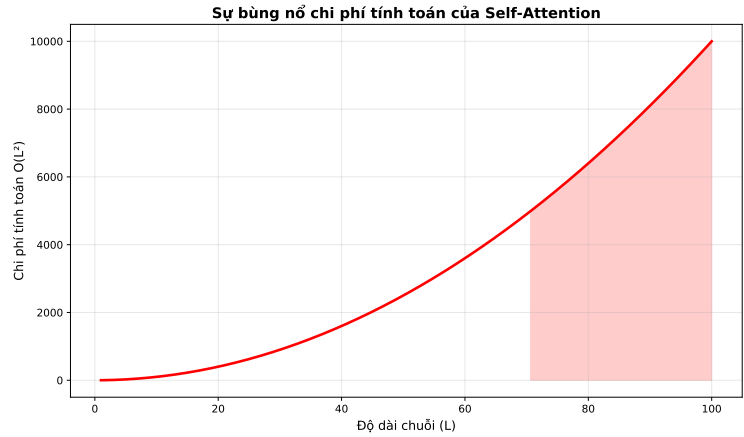
Hình 1: Sự biến động phức tạp và “nhiều” của thị trường chứng khoán, một thách thức lớn cho các mô hình dự báo dài hạn.

Để giải quyết bài toán LTSF một cách hiệu quả, mô hình không chỉ đơn thuần là mở rộng của sổ dự báo. Thách thức cốt lõi nằm ở việc nắm bắt các phụ thuộc dài hạn (long-term dependencies). Trong dữ liệu tài chính, một sự kiện ở thời điểm t (ví dụ: giá hôm nay) có thể không chỉ phụ

thuộc vào $t - 1$ (hôm qua), mà còn bị ảnh hưởng mạnh mẽ bởi các mô hình (patterns) đã xảy ra từ nhiều tháng, hoặc thậm chí nhiều năm trước. Ví dụ, tính mùa vụ (seasonality) có thể tạo ra các liên hệ cách nhau 12 tháng, hoặc các chu kỳ kinh tế (market cycles) có thể tạo ra các phụ thuộc kéo dài 3-5 năm. Các phương pháp dự báo truyền thống, thường thất bại trong việc “ghi nhớ” và liên kết các thông tin ở quá khứ xa xôi này. Do đó, yêu cầu cấp thiết là phải có một kiến trúc có khả năng “nhìn” và kết nối hiệu quả các điểm dữ liệu bất kể chúng cách xa nhau bao nhiêu, cho phép mô hình phân biệt được tín hiệu dài hạn thực sự khỏi nhiễu cục bộ.

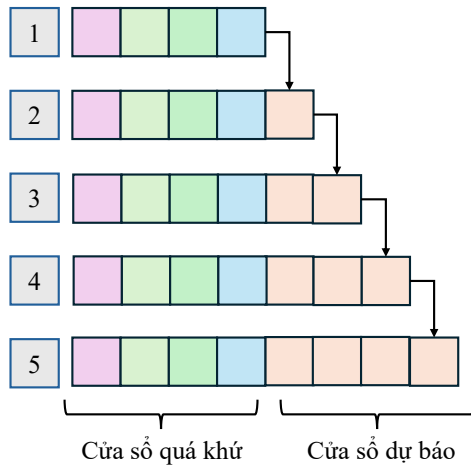
Khi nghe đến vấn đề phụ thuộc dài hạn, chúng ta thường nghĩ ngay đến lớp mô hình Transformer. Với cơ chế self-attention, các mô hình như Informer hay Autoformer hứa hẹn khả năng nhìn toàn bộ dòng lịch sử để tìm ra các mối quan hệ, phức tạp.

Nhưng có một vấn đề lớn với mô hình Transformer chính là độ phức tạp $O(L^2)$ của thuật toán. Khi L là độ dài chuỗi đầu vào (ví dụ $L = 720$, tương đương dữ liệu giao dịch 3 năm theo ngày), chi phí tính toán và bộ nhớ sẽ tăng vọt. Điều này khiến việc huấn luyện các mô hình Transformer cho chuỗi rất dài trở nên cực kỳ tốn kém, thậm chí là không khả thi trong thời gian thật.

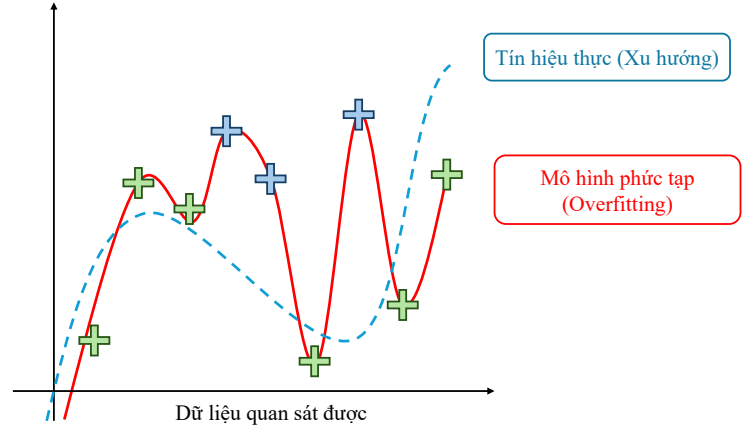


Hình 2: Sự bùng nổ về chi phí tính toán của cơ chế self-attention. Khi độ dài chuỗi L tăng lên, chi phí $O(L^2)$ khiến mô hình trở nên không thực tế cho các tác vụ dự báo rất dài. Vùng màu đỏ là khu vực mà VRAM sử dụng lớn hơn 8GB.

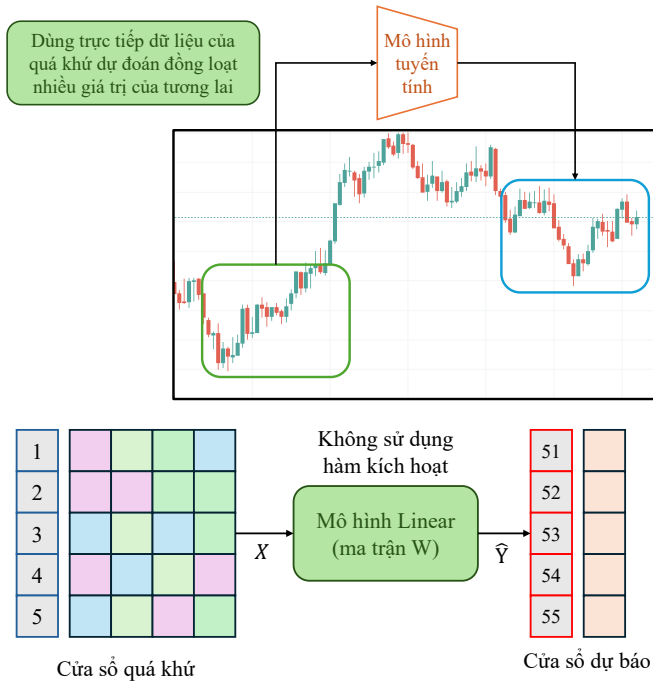
Điều này đặt ra một câu hỏi then chốt, đặc biệt là với dữ liệu nhiều nhiễu như tài chính: Liệu một kiến trúc siêu phức tạp có đang vô tình ‘học vẹt’ (overfit) chính cái nhiễu đó, thay vì nắm bắt tín hiệu thực sự?



Hình 3: Các mô hình chuỗi thời gian thường dự đoán tương lai bằng cách dự đoán tuần tự $t, t+1, \dots, t+n$.



Hình 4: Minh họa hiện tượng Overfitting trong dữ liệu chuỗi thời gian. Các mô hình phức tạp thường ‘học vẹt’ một cách hoàn hảo các dao động ngẫu nhiên (nhiều) có trong dữ liệu.



Hình 5: Kiến trúc của mô hình Linear. Thay vì dự đoán tuần tự các ngày trong tương lai, chúng chỉ cần chạy mô hình một lần.

Giữa bối cảnh đó, một nhóm mô hình ‘quay về cơ bản’ (back-to-basics) mang tên LTSF-Linear xuất hiện. Ý tưởng của chúng đơn giản đến bất ngờ: hãy mô hình hóa toàn bộ trực thời gian chỉ bằng một phép ánh xạ tuyến tính.

- Linear: Mô hình cơ sở, ‘kéo’ trực tiếp cửa sổ quá khứ sang tương lai bằng một ma trận trọng số duy nhất.
- DLinear: Cải tiến bằng cách phân rã (decompose) tín hiệu thành xu thế (trend) và dao động (seasonality), sau đó dùng hai mô hình Linear riêng biệt.
- NLinear: Bổ sung bước chuẩn hóa (normalization) để giảm lệch phân phối (distribution shift) trước khi đưa vào mô hình Linear.

Đặc biệt, trong các benchmark, những mô hình tuyến tính đơn giản này không chỉ nhẹ và ổn định, mà còn cho ra kết quả cạnh tranh, thậm chí vượt trội so với các Transformer phức tạp. Điều này dấy lên một giả thuyết mạnh mẽ: trong một miền ‘nhiều’ như tài chính, việc tách lọc

tín hiệu (như DLinear làm) có thể còn quan trọng hơn là cố gắng mô hình hóa mọi mối quan hệ phức tạp có thể có trong không gian bài toán.

Mục tiêu project: Project này không chỉ là một bài hướng dẫn để ‘chạy lại’ code mà còn là một project cơ sở để bạn hiểu việc xử lý dữ liệu chuỗi thời gian và đồng thời cải tiến chúng. Nhiệm vụ của bạn là:

- Hiểu rõ cách triển khai và so sánh Linear, NLinear, và DLinear trên dữ liệu tài chính thực tế.
- Phân tích (và phản biện) kết quả: Tại sao một mô hình đơn giản lại có thể hoạt động tốt? Giới hạn của nó ở đâu?
- Thử nghiệm và cải tiến: Bạn sẽ làm gì tiếp theo để cải thiện hiệu suất dự báo từ cái nền tảng này? (Ví dụ: thay đổi cửa sổ look-back, thử nghiệm với các nhóm cổ phiếu khác nhau, hay kết hợp các ý tưởng?)

Project này được thiết kế cho các bạn:

- Học viên AI/data science: Muốn áp dụng và ‘thách thức’ các mô hình time series trên dữ liệu tài chính phức tạp.
- Nhà phân tích định lượng (quants): Cần hiểu rõ các mô hình baseline mạnh, nhẹ để làm nền tảng cho các hệ thống phức tạp hơn.
- Bất kỳ ai tò mò: Giữa các lớp mô hình ‘đơn giản’ và ‘phức tạp’ trong machine learning, có nhất thiết càng phức tạp sẽ càng tốt hơn không?

Mục lục

I.	Giới thiệu	1
II.	Kiến thức cơ bản về dự báo chuỗi thời gian	7
II.1.	Bài toán dự báo chuỗi thời gian là gì?	7
II.2.	Dữ liệu trông như thế nào?	8
II.3.	Chúng ta muốn dự đoán cái gì?	9
II.4.	Liên hệ tới các mô hình trong project này	10
III.	Mô hình LTSF-Linear: Linear	10
IV.	Mô hình LTSF-Linear: NLinear	11
V.	Mô hình LTSF-Linear: DLinear	13
VI.	Xây dựng các mô hình LTSF-Linear từ đầu (from sratch)	16
VII.	Bài toán dự đoán giá cổ phiếu sử dụng LTSF-Linear Models	20
VIII.	Hướng cải tiến và mở rộng	36
VIII.1.	Mở rộng sang bài toán đa biến	36
VIII.2.	Hybrid model: kết hợp phân rã và transformer	36
IX.	Câu hỏi trắc nghiệm	38
	Phụ lục	40

Để việc đọc thuận tiện hơn, bảng sau liệt kê và chuẩn hoá các ký hiệu sẽ xuất hiện xuyên suốt nội dung bài.

Bảng Ký hiệu Toán học (LTSF)

Ký hiệu	Ý nghĩa
L	Độ dài cửa sổ quá khứ.
T	Độ dài dự báo tương lai.
B	Kích thước một batch.
$\mathbf{x} \in \mathbb{R}^L$	Vector input, chứa L giá trị quá khứ.
$\mathbf{y} \in \mathbb{R}^T$	Vector mục tiêu, chứa T giá trị thực tương lai.
$\hat{\mathbf{y}} \in \mathbb{R}^T$	Vector dự đoán, chứa T giá trị dự đoán tương lai.
x_t	Giá trị của chuỗi thời gian tại thời điểm t .
y_t	Giá trị thực tại bước tương lai thứ t .
$\hat{y}_t$	Giá trị dự đoán tại bước tương lai thứ t .
$W \in \mathbb{R}^{T \times L}$	Mã trận trọng số của mô hình Linear.
$b \in \mathbb{R}^T$	Vector bias của mô hình Linear.
ℓ	Mốc cục bộ (giá trị cuối x_t của $\mathbf{x}$) để tịnh tiến.
$\mathbf{1}_L, \mathbf{1}_T$	Vector chỉ chứa số 1, với độ dài L và T .
$\mathbf{x}'$	Vector input $\mathbf{x}$ đã được tịnh tiến (trừ ℓ).
$\hat{\mathbf{y}}'$	Vector dự đoán $\hat{\mathbf{y}}$ trước khi hoàn nguyên (cộng ℓ).
m	Kích thước cửa sổ của Moving Average.
$\text{MA}_m(\cdot)$	Toán tử moving average với cửa sổ m .
x_t	Thành phần xu hướng của $\mathbf{x}$.
x_s	Thành phần mùa vụ của $\mathbf{x}$.
W_t, b_t	Mã trận trọng số và bias cho nhánh xu hướng.
W_s, b_s	Mã trận trọng số và bias cho nhánh mùa vụ.
$\hat{y}_t, \hat{y}_s$	Vector dự đoán cho nhánh Trend và Seasonal.

II. Kiến thức cơ bản về dự báo chuỗi thời gian

Trước khi đi vào mô hình, chúng ta đi qua cách đóng gói dữ liệu chuỗi thời gian để đưa vào mô hình học máy, kèm các khái niệm nền tảng về loại bài toán, đầu vào/đầu ra, và cách đánh giá các mô hình trên bài toán này.

II.1. Bài toán dự báo chuỗi thời gian là gì?

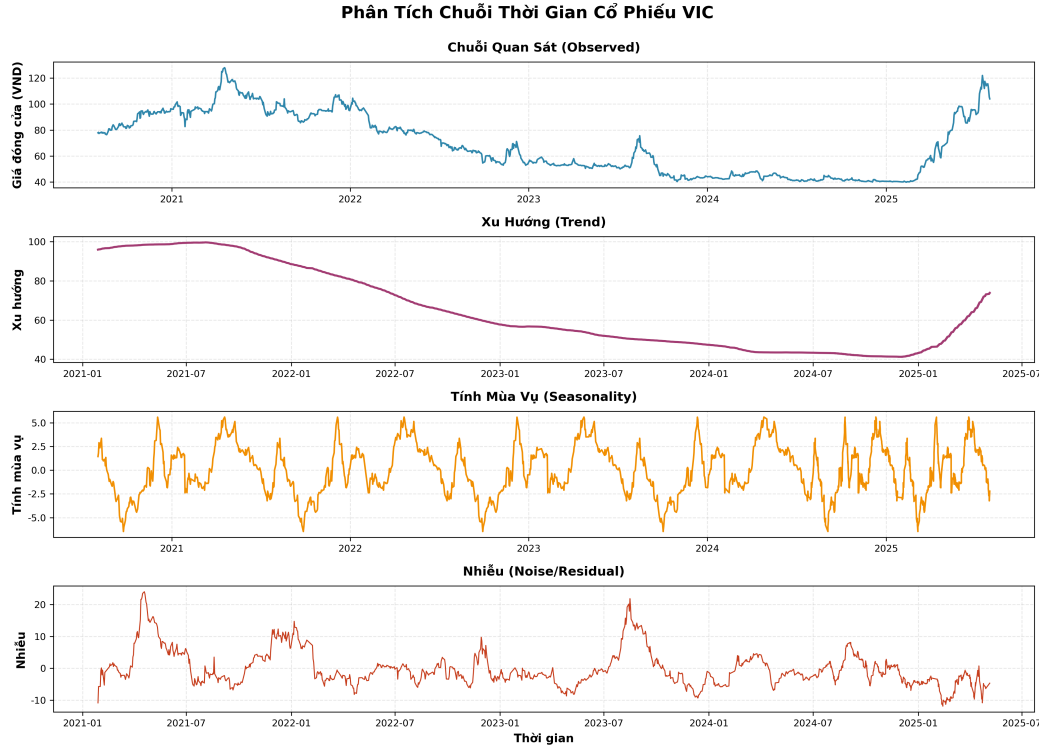
Một chuỗi thời gian đơn giản là một chuỗi các điểm dữ liệu được ghi lại theo thứ tự thời gian, ví dụ như giá cổ phiếu mỗi ngày, hoặc nhiệt độ mỗi giờ. Mục tiêu của bài toán dự báo là: dùng dữ liệu quá khứ để dự đoán dữ liệu tương lai. Chúng ta gọi độ dài của quá khứ mà mô hình nhìn vào là cửa sổ quá khứ (Look-back, L), và số bước tương lai cần dự đoán là chân trời dự đoán (Horizon, H).

Một cách tổng quát, chúng ta đang cố gắng tìm một hàm dự báo để học được quy luật này:

$$\underbrace{y_{t+1:t+H}}_{\text{đầu ra}} = \text{hàm dự báo} \left(\underbrace{y_{t-L+1:t}}_{\text{quá khứ}}, \underbrace{x_{t-L+1:t}}_{\text{biến phụ trợ đã quan sát}}, \underbrace{x_{t+1:t+H}}_{\text{biến phụ trợ biết trước}}, \underbrace{s}_{\text{đặc trưng tĩnh}} \right) + \underbrace{\varepsilon}_{\text{nhiều}}.$$

Công thức này trông phức tạp, nhưng nó chỉ mô tả những gì chúng ta đã nói. Để dự đoán chính xác hơn (đầu ra màu xanh), chúng ta không chỉ dùng giá trị trong quá khứ (màu đỏ). Chúng ta cần cung cấp thêm thông tin, gọi là **biến phụ trợ** (covariates). Các loại này bao gồm **biến phụ trợ đã quan sát** (màu xanh lá), như lượng mưa hôm qua. Chúng cũng bao gồm các **biến phụ trợ biết trước** (màu tím), như ngày mai là chủ nhật. Và cuối cùng là các **đặc trưng tĩnh** (màu cam), như id của cửa hàng.

Khi nhìn vào một chuỗi thời gian, chúng ta thường có thể phân tách nó thành các thành phần. Các thành phần chính bao gồm **xu hướng (trend)**, là hướng đi chung của dữ liệu; **mùa vụ (seasonality)**, là các mẫu lặp lại có chu kỳ; và **nhiều (noise)**, là các biến động ngẫu nhiên. Các mô hình khác nhau sẽ có cách xử lý các thành phần này khác nhau.

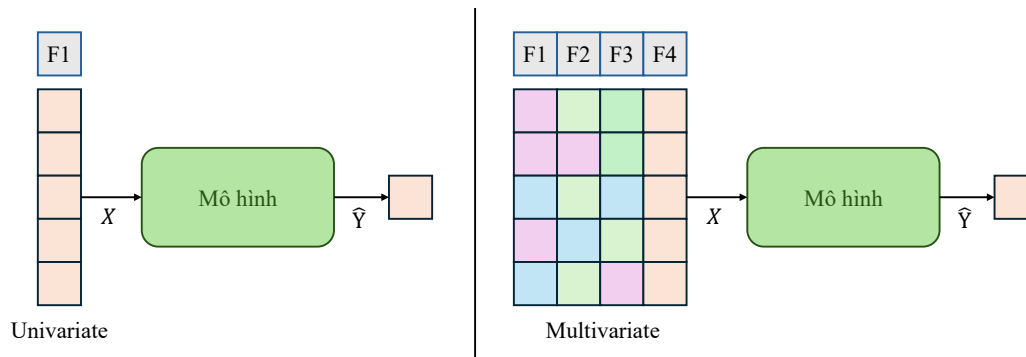


Hình 6: Phân tích chuỗi thời gian chúng ta sẽ sử dụng trong project này. Ta thấy rằng bài toán chúng ta có rất nhiều nhiễu, như ở plot cuối cùng có nhiều đường lên xuống.

II.2. Dữ liệu trông như thế nào?

Về cấu trúc, dữ liệu đầu vào có thể là đơn biến (univariate), nghĩa là chúng ta chỉ có một đặc trưng duy nhất (ví dụ: chỉ dùng chuỗi giá đóng cửa của cổ phiếu A). Hoặc, dữ liệu có thể là đa biến (multivariate), khi chúng ta sử dụng nhiều đặc trưng đầu vào cùng một lúc (ví dụ: dùng giá đóng cửa, khối lượng giao dịch của cổ phiếu A, và cả chỉ số VN-Index) để đưa ra dự đoán.

Một khái niệm liên quan là cấu trúc đầu ra (output). Chúng ta có thể dự đoán một chuỗi (ví dụ: chỉ dự đoán giá cổ phiếu A), hoặc dự đoán nhiều chuỗi cùng lúc (ví dụ: dự đoán giá của A, B, và C). Hình dưới minh họa sự khác biệt về cấu trúc đầu vào.

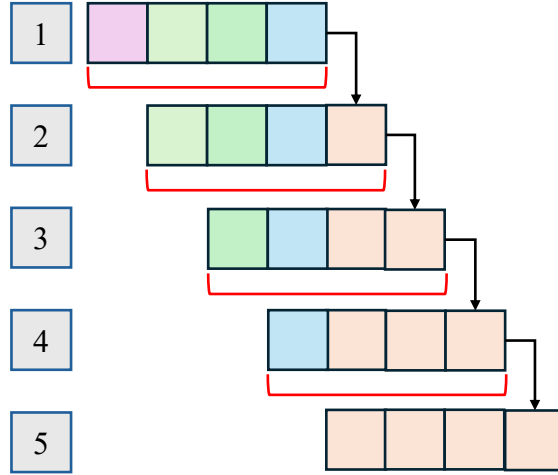


Hình 7: Hai cách biểu diễn đầu vào của bài toán chuỗi thời gian. Trong đó, cột cam là cột dữ liệu chứa các biến cần dự đoán.

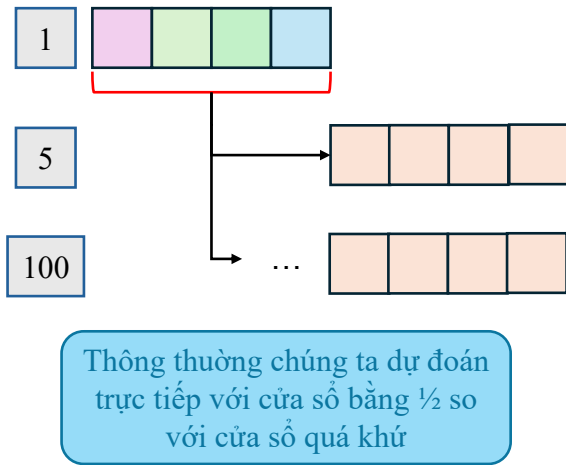
II.3. Chúng ta muốn dự đoán cái gì?

Khi nói về kiểu dự báo, chúng ta có thể dự đoán 1 bước (one-step) là giá trị ngay tiếp theo, hoặc **dự đoán nhiều bước (multi-step)** là một chuỗi giá trị trong tương lai ($H > 1$). Đây là mục tiêu chính của chúng ta ở trong bài toán này.

Chiến lược phổ biến thứ nhất cho bài toán multi-step là **đệ quy (recursive)**. Theo cách này, mô hình đầu tiên dự đoán bước thời gian $t + 1$. Sau đó, kết quả dự đoán (giả) này được sử dụng làm đầu vào mới để mô hình tiếp tục dự đoán bước $t + 2$. Quá trình này lặp đi lặp lại cho đến khi đủ H bước. Mặc dù trực quan, phương pháp này có một rủi ro rất lớn là tích lũy lỗi (compounding errors): một lỗi nhỏ ở bước $t + 1$ có thể bị khuếch đại và làm sai lệch nghiêm trọng các dự đoán ở $t + H$.



Hình 8: Chiến lược dự đoán đệ quy: Mô hình dự đoán từng bước một, sử dụng kết quả dự đoán trước đó làm đầu vào cho bước tiếp theo.



Hình 9: Chiến lược dự đoán trực tiếp: Mô hình ánh xạ toàn bộ quá khứ đến toàn bộ tương lai trong một lần, tránh tích lũy lỗi.

Chiến lược thứ hai, hiệu quả hơn là **trực tiếp (direct)**. Với chiến lược này, mô hình được huấn luyện để học cách ánh xạ thẳng từ toàn bộ cửa sổ dữ liệu quá khứ (X) để dự đoán đồng thời *toàn bộ* H bước tương lai ($\hat{Y}$) chỉ trong một lần duy nhất. Phương pháp này hoàn toàn tránh được vấn đề tích lũy lỗi, vì mỗi dự đoán đều dựa trên dữ liệu thực tế. Đây chính là chiến lược cốt lõi được các mô hình Linear, NLinear, và DLinear sử dụng.

Cuối cùng, kết quả dự báo có thể là một **dự đoán điểm (point forecast)**, là một con số duy nhất như dự đoán ngày mai giá là 100. Hoặc, nó có thể là một **dự đoán xác suất (probabilistic forecast)**, trả về một khoảng giá trị, như dự đoán ngày mai giá nằm trong khoảng 95-105 với độ chắc chắn 90%.

II.4. Liên hệ tới các mô hình trong project này

Các mô hình trong project này đều áp dụng chiến lược dự báo trực tiếp. Mô hình **NLinear** tập trung vào việc chuẩn hóa (normalization) dữ liệu đầu vào để xử lý vấn đề khi dữ liệu trong tương lai trông khác nhiều so với quá khứ. Mô hình **DLinear** thì thực hiện theo một cách khác: nó tự động tách chuỗi thời gian thành hai phần là xu hướng và mùa vụ. Sau đó, nó dùng hai mô hình linear riêng biệt để học hai phần này, hoạt động rất tốt cho dữ liệu có xu hướng và mùa vụ rõ rệt.

III. Mô hình LTSF-Linear: Linear

Cơ chế: Linear mô hình hoá dự đoán dài hạn như một phép biến đổi tuyến tính dọc trục thời gian cho một biến (univariate). Với mỗi mẫu trong batch, ta có input $\mathbf{x} \in \mathbb{R}^L$ (cửa sổ quá khứ L bước) và output $\mathbf{y} \in \mathbb{R}^T$ (dự đoán T bước). Mô hình học ma trận $W \in \mathbb{R}^{T \times L}$ và bias $b \in \mathbb{R}^T$:

$$\hat{\mathbf{y}} = W \mathbf{x} + b \in \mathbb{R}^T.$$

Huấn luyện (DMS - Direct Multi-step). Tối ưu *đồng thời* cả T bước bằng MSE:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{BT} \sum_{n=1}^B \sum_{t=1}^T (\hat{y}_t - y_t)^2.$$

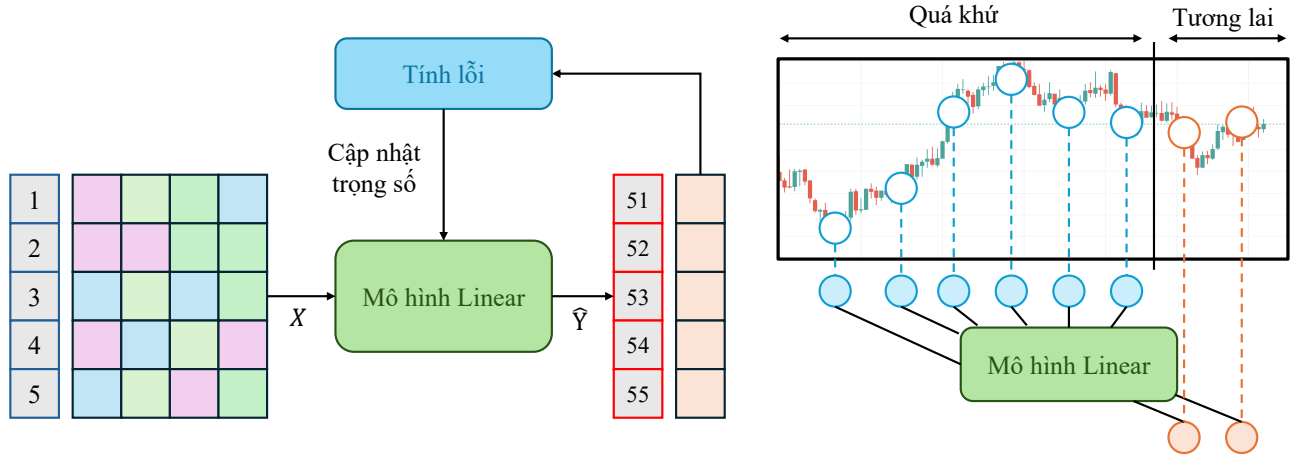
Trong đó:

- T : số bước dự đoán (horizon); B : kích thước batch.
- $\hat{y}_t$: giá trị **dự đoán** tại bước tương lai t ; y_t : giá trị **thực**.
- MSE là **trung bình** sai số bình phương trên toàn bộ T bước và B mẫu.

Độ phức tạp. Tham số $\approx T \times L$ (cộng T bias); chi phí tính toán tỉ lệ $B \times T \times L \Rightarrow$ huấn luyện/suy luận nhanh, ổn định.

Lưu ý

Mô hình Linear trực tiếp biến đổi tuyến tính từ L bước gần nhất để dự đoán T bước kế tiếp, phù hợp khi chuỗi chúng ta đang xét ổn định. Nếu chuỗi có thuộc tính phi tuyến mạnh, có nhiều thay đổi đột ngột, hoặc cần mô hình hoá quan hệ đa biến, cân nhắc NLinear/DLinear hoặc các mô hình phi tuyến khác.



Hình 10: Quy trình chạy và huấn luyện của mô hình LTSF-Linear.

IV. Mô hình LTSF-Linear: NLinear

Cơ chế: Mô hình NLinear mở rộng lớp Linear cho dự đoán đơn biến bằng bước tịnh tiến dữ liệu về mốc cục bộ (**re-centering**) theo giá trị quan sát cuối của cửa sổ đang dự đoán, nhằm làm mô hình tập trung học sự biến thiên tương đối và giảm nhạy cảm trước dịch mức cộng tính (**additive level shift**). Hai thuật ngữ khó hiểu nói trên sẽ được diễn giải dưới đây.

Thuật ngữ:

- **Additive level shift:** toàn bộ các giá trị trong một đoạn thời gian cùng tăng hoặc cùng giảm một lượng như nhau. Nói cách khác, “mặt bằng” của chuỗi được đẩy lên hoặc hạ xuống nhưng hình dáng dao động (nhịp lên xuống, độ chênh giữa các điểm liên kề) hầu như không đổi.
- **Re-centering:** chọn một mốc tham chiếu gần nhất (thường là giá trị cuối của sổ), trừ mốc này khỏi mọi giá trị trong cửa sổ để đưa chuỗi về quanh mốc đó, huấn luyện mô hình trên phần độ lệch tương đối, rồi cộng lại mốc tham chiếu vào đầu ra. Cách này giúp mô hình ít bị ảnh hưởng khi mặt bằng dữ liệu dịch lên/xuống đồng đều.

Thiết lập dữ liệu. Với chuỗi $\{x_t\}$, tại thời điểm t tạo

$$\mathbf{x} = [x_{t-L+1}, \dots, x_t] \in \mathbb{R}^L, \quad \mathbf{y} = [x_{t+1}, \dots, x_{t+T}] \in \mathbb{R}^T.$$

Trong đó (thiết lập):

- $\mathbf{x}$: input quá khứ dài L bước; $\mathbf{y}$: mục tiêu dự đoán T bước tương lai.
- $\mathbf{1}_L, \mathbf{1}_T$: vectơ với mọi phần tử đều bằng 1, kích thước L và T .

Công thức. Đặt mốc cục bộ $\ell = x_t$, thực hiện ba bước: *tịnh tiến* $\rightarrow$ *chiếu tuyến tính* $\rightarrow$ *hoàn*

nguyên:

$$\mathbf{x}' = \mathbf{x} - \ell \mathbf{1}_L \quad (\text{tính tiến: đưa toàn bộ giá trị trong cửa sổ về quanh mốc cuối } \ell)$$

$$\hat{\mathbf{y}}' = W \mathbf{x}' + b$$

$$\hat{\mathbf{y}} = \hat{\mathbf{y}}' + \ell \mathbf{1}_T \in \mathbb{R}^T \quad (\text{hoàn nguyên: cộng lại mốc } \ell \text{ để trả dự đoán về mức gốc})$$

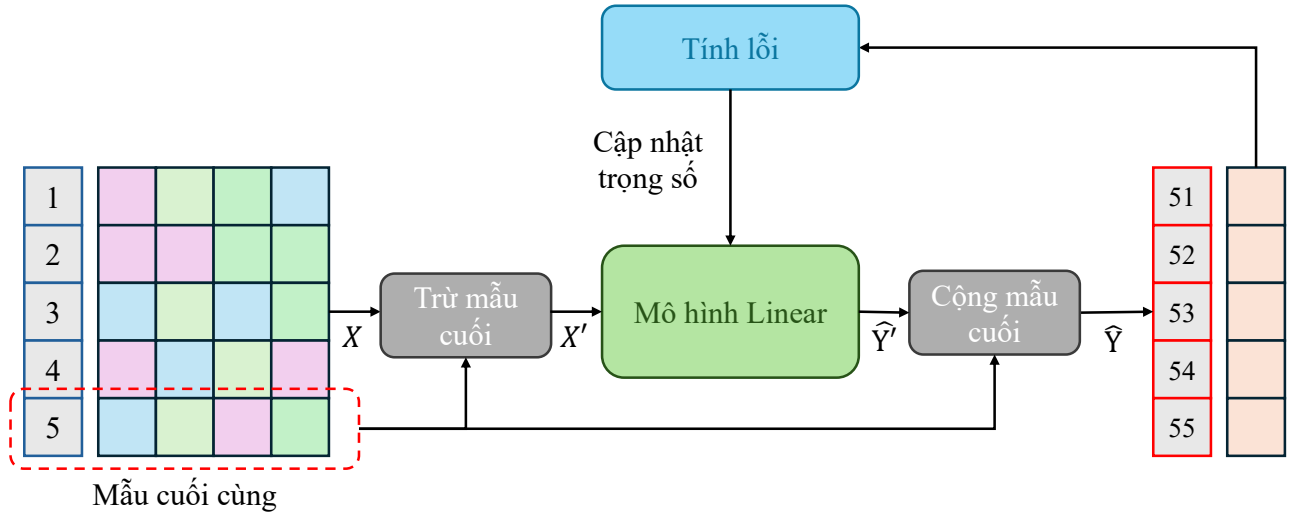
Huấn luyện (DMS). Tối ưu đồng thời cả T bước bằng MSE trên batch B :

$$\mathcal{L}_{\text{MSE}} = \frac{1}{BT} \sum_{n=1}^B \sum_{t=1}^T (\hat{y}_t - y_t)^2.$$

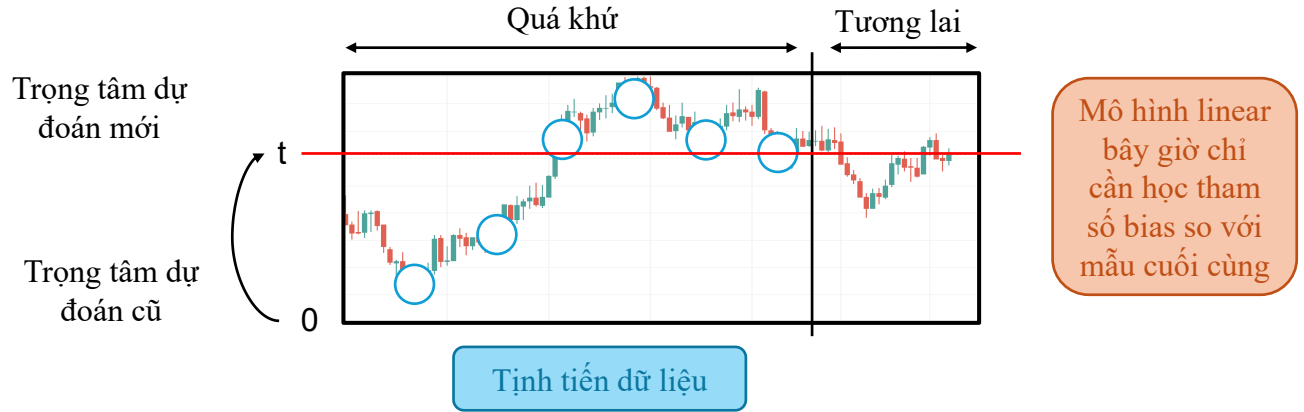
Trong đó (huấn luyện):

- T : số bước dự đoán (horizon); B : kích thước batch.
- $\hat{y}_t, y_t$: giá trị dự đoán và giá trị thực tại bước tương lai t .
- MSE: trung bình sai số bình phương trên toàn bộ T bước và B mẫu (tối ưu trực tiếp, không cuộn từng bước).

Độ phức tạp. Như Linear: tham số $\approx T \times L$ (cộng T bias); chi phí $\propto B \times T \times L$.



Hình 11: Quy trình chạy và huấn luyện của mô hình LTSF-NLinear.



Hình 12: Khi chúng ta biến đặt mốc cực bộ giá trị cuối cùng, mô hình Linear sẽ cần phải học ít thông tin hơn để đưa ra kết quả.

V. Mô hình LTSF-Linear: DLinear

Cơ chế: DLinear (đơn biến) *phân rã* chuỗi thành **trend** và **seasonal/residual**, sau đó *chiều tuyến tính* riêng cho từng phần và *cộng* lại. Cách làm này phù hợp khi dữ liệu có trend và/hoặc seasonal rõ rệt.

Ví dụ về $MA_m(\cdot)$ (moving average-cách tạo trend):

Giả sử $m=3$ với xử lý rìa kiểu **replicate** (tạm dịch là: lặp biên). Với $\mathbf{x} = [10, 12, 11, 13]$:

$$MA_3(\mathbf{x}) = \left[\frac{10+10+12}{3}, \frac{10+12+11}{3}, \frac{12+11+13}{3}, \frac{11+13+13}{3} \right] = [10.67, 11.00, 12.00, 12.33].$$

Chuỗi trên đóng vai trò *trend*; phần *seasonal/residual* là $\mathbf{x} - MA_3(\mathbf{x})$.

Thuật ngữ:

- **trend:** thành phần thay đổi chậm theo thời gian, phản ánh xu hướng dài hạn của chuỗi.
- **seasonal/residual:** phần còn lại sau khi bỏ trend; thường chứa nhịp lặp theo chu kỳ và dao động nhanh.

Thiết lập dữ liệu. Với chuỗi $\{x_t\}$, tại thời điểm t tạo

$$\mathbf{x} = [x_{t-L+1}, \dots, x_t] \in \mathbb{R}^L, \quad \mathbf{y} = [x_{t+1}, \dots, x_{t+T}] \in \mathbb{R}^T.$$

Trong đó (thiết lập):

- $\mathbf{x}$: input quá khứ dài L ; $\mathbf{y}$: mục tiêu T bước tương lai.
- $\mathbf{1}_L, \mathbf{1}_T$: vectơ toàn 1 kích thước L và T .

Công thức. Chọn bậc làm mượt m (ví dụ $m=5$). Phân rã bằng moving average, dự đoán tuyến tính theo từng phần, rồi cộng:

$$\begin{aligned}
 x_t &= \text{MA}_m(x) && \text{trend (moving average của } x) \\
 x_s &= x - x_t && \text{seasonal/residual (phần còn lại)} \\
 \hat{y}_t &= W_t x_t + b_t && \text{linear head cho trend } (W_t \in \mathbb{R}^{T \times L}, b_t \in \mathbb{R}^T) \\
 \hat{y}_s &= W_s x_s + b_s && \text{linear head cho seasonal/residual } (W_s \in \mathbb{R}^{T \times L}, b_s \in \mathbb{R}^T) \\
 \hat{y} &= \hat{y}_{\text{trend}} + \hat{y}_{\text{seasonal}} \in \mathbb{R}^T && \text{tổng hợp (forecast cuối cùng)}
 \end{aligned}$$

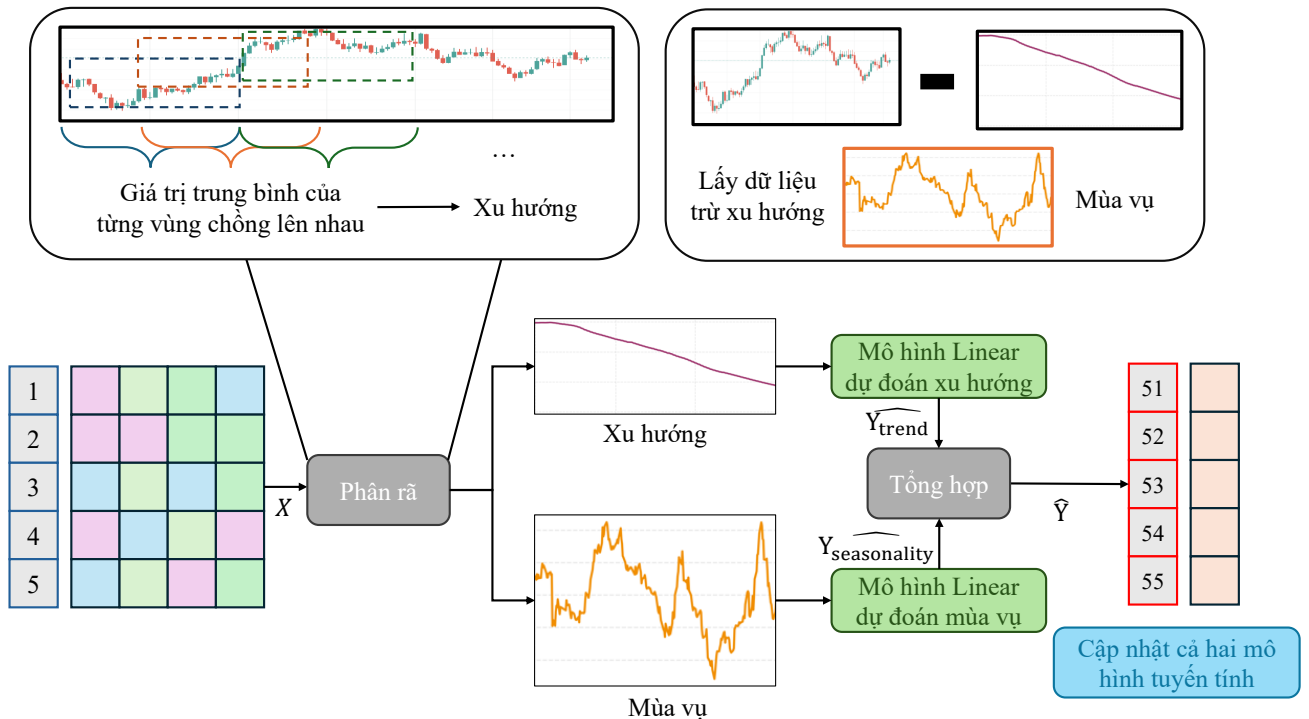
Huấn luyện (DMS). Tối ưu đồng thời cả T bước bằng MSE trên batch B :

$$\mathcal{L}_{\text{MSE}} = \frac{1}{BT} \sum_{n=1}^B \sum_{t=1}^T (\hat{y}_t - y_t)^2.$$

Trong đó (huấn luyện):

- T : số bước dự đoán (horizon); B : kích thước batch.
- $\hat{y}_t, y_t$: giá trị dự đoán và thực tại bước t .
- MSE: trung bình sai số bình phương trên toàn bộ T bước và B mẫu (tối ưu trực tiếp, không cuộn từng bước).

Độ phức tạp. Hai đầu tuyến tính nên tham số $\approx 2(T \times L)$ (cộng $2T$ bias); chi phí $\propto B \times T \times L$ (nhân đôi hằng số do hai nhánh), vẫn rất nhẹ.



Hình 13: Quy trình chạy và huấn luyện của mô hình LTSF-DLinear.

i Lưu ý

Trong DLinear, tham số m biểu thị độ dài cửa sổ trung bình trượt (moving average window length), tức số điểm dữ liệu được lấy trung bình để ước lượng xu hướng $\text{Trend}(x_t)$. Tham số này xác định mức độ làm mượt của *low-pass filter* — bộ lọc cho phép thành phần tần số thấp (xu hướng dài hạn) đi qua và loại bỏ thành phần tần số cao (dao động ngắn hạn). Khi m lớn, xu hướng trở nên trơn hơn nhưng làm giảm chi tiết chu kỳ; ngược lại, m nhỏ giúp phản ứng nhanh hơn với biến động ngắn hạn. DLinear phù hợp với chuỗi có xu hướng hoặc chu kỳ rõ, trong khi NLinear hiệu quả hơn với dữ liệu có *level shift*, tức là các điểm thời gian mà giá trị kỳ vọng (hoặc trung bình cục bộ) của chuỗi thay đổi đột ngột.

VI. Xây dựng các mô hình LTSF-Linear từ đầu (from sratch)

Để minh họa rõ cơ chế hoạt động, ta xây dựng lại ba mô hình **Linear**, **NLinear** và **DLinear** với phiên bản đơn giản, dễ hiểu hơn từ đầu.

Bước 1: Import thư viện và tạo dữ liệu mẫu

```

1 import torch
2 import torch.nn as nn
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import pandas as pd
6
7 data = torch.tensor([[100.0, 102.0, 98.0, 105.0, 103.0]]) # [1, 5]
8 print(f"Input: {data.squeeze().numpy()}")
9
10 target = torch.tensor([[101.0, 104.0, 106.0]]) # [1, 3]
11 print(f"Target: {target.squeeze().numpy()}")

```

Ở bước này, bạn đã có dữ liệu đầu vào $\mathbf{x} \in \mathbb{R}^{1 \times 5}$ (5 bước quá khứ) và mục tiêu $\mathbf{y} \in \mathbb{R}^{1 \times 3}$ (3 bước cần dự báo). Ta sẽ dùng chúng để kiểm tra mô hình ở các bước sau.

Bước 2: Xây dựng mô hình Linear với trọng số cố định

Code Exercise

```

1 class Linear(nn.Module):
2     def __init__(self):
3         super().__init__()
4         self.linear = nn.Linear(5, 3)
5         # Set weights cụ thể
6         with torch.no_grad():
7             self.linear.weight.data = torch.tensor([
8                 [0.2, 0.3, -0.1, 0.4, 0.2], # Output 1
9                 [0.1, 0.2, 0.3, 0.3, 0.1], # Output 2
10                [0.0, 0.1, 0.2, 0.4, 0.3]   # Output 3
11            ])
12            self.linear.bias.data = torch.tensor([1.0, 2.0, 3.0])
13
14     def forward(self, x):
15         return ***Your Code Here***
16
17 # Test Linear
18 linear_model = Linear()

```

```

19 linear_out = ***Your Code Here***
20
21 print("LINEAR MODEL:")
22 print(f"   Weights:\n{linear_model.linear.weight.data.numpy()}")
23 print(f"   Bias: {linear_model.linear.bias.data.numpy()}")
24 print(f"   Input: {data.squeeze().numpy()}")
25 print(f"   Output: {linear_out.squeeze().detach().numpy()}")
26
27 # Manual calculation để verify
28 print("   Manual calc:")
29 for i in range(3):
30     w = linear_model.linear.weight.data[i]
31     b = linear_model.linear.bias.data[i]
32     result = torch.sum(w * data.squeeze()) + b
33     print(f"       Out{i+1}: {result:.3f}")

```

Thực hành điền code vào các vị trí *****Your Code Here***** để hoàn thành các yêu cầu sau:

- Hoàn thiện hàm `forward` để đưa đầu vào qua lớp `nn.Linear` và nhận đầu ra có kích thước phù hợp.
- Gọi mô hình với `data` để lấy `linear_out`.
- Đối chiếu Output với phần tính tay (Manual calculation) để kiểm tra công thức tuyến tính $\hat{y}_i = \sum_j W_{ij}x_j + b_i$.

Bước 3: Xây dựng mô hình DLinear với trọng số cố định

Code Exercise

```

1 class DLinear(nn.Module):
2     def __init__(self):
3         super().__init__()
4         self.linear_trend = nn.Linear(5, 3)
5         self.linear_seasonal = nn.Linear(5, 3)
6
7         # Set weights cụ thể cho trend
8         with torch.no_grad():
9             self.linear_trend.weight.data = torch.tensor([
10                 [0.3, 0.3, 0.2, 0.1, 0.1],
11                 [0.2, 0.3, 0.3, 0.1, 0.1],
12                 [0.1, 0.2, 0.3, 0.3, 0.1]
13             ])
14             self.linear_trend.bias.data = torch.tensor([0.5, 1.0, 1.5])
15
16         # Set weights cho seasonal
17         self.linear_seasonal.weight.data = torch.tensor([
18             [0.1, -0.2, 0.3, -0.1, 0.0],
19             [0.0, 0.1, -0.2, 0.2, 0.1],
20             [-0.1, 0.0, 0.1, 0.1, 0.2]

```

```

21         ])
22         self.linear_seasonal.bias.data = torch.tensor([0.2, 0.3, 0.1])
23
24     def decompose(self, x):
25         # Simple moving average (window=3) - replicate padding
26         x_np = x.squeeze().numpy()
27         trend = []
28         for i in range(len(x_np)):
29             if i == 0:
30                 ***Your Code Here***
31             elif i == len(x_np) - 1:
32                 ***Your Code Here***
33             else:
34                 ***Your Code Here***
35             trend.append(avg)
36
37         trend_tensor = torch.tensor(trend).unsqueeze(0)
38         seasonal = ***Your Code Here***
39         return trend_tensor, seasonal
40
41     def forward(self, x):
42         trend, seasonal = ***Your Code Here***
43         trend_pred = self.linear_trend(trend)
44         seasonal_pred = self.linear_seasonal(seasonal)
45         return ***Your Code Here***
46 # Test DLinear
47 dlinear_model = DLinear()
48 dlinear_out = dlinear_model(data)
49
50 print("DLINER MODEL:")
51 trend, seasonal = dlinear_model.decompose(data)
52 print(f"    Input: {data.squeeze().numpy()}")
53 print(f"    Trend: {trend.squeeze().numpy()}")
54 print(f"    Seasonal: {seasonal.squeeze().numpy()}")
55 print(f"    Output: {dlinear_out.squeeze().detach().numpy()}")
56
57 # Show component predictions
58 trend_pred = dlinear_model.linear_trend(trend)
59 seasonal_pred = dlinear_model.linear_seasonal(seasonal)
60 print(f"    Trend pred: {trend_pred.squeeze().detach().numpy()}")
61 print(f"    Seasonal pred: {seasonal_pred.squeeze().detach().numpy()}")

```

Thực hành điền code vào các vị trí *****Your Code Here***** để hoàn thành các yêu cầu sau:

- Trong `decompose`: áp dụng trung bình trượt cửa sổ 3 điểm với replicate padding ở hai đầu để tạo dãy trend; sau đó lấy tín hiệu gốc trừ trend tại từng vị trí để thu được seasonal.
- Trong `forward`: gọi `decompose` để lấy trend và seasonal; đưa trend qua `linear_trend`, đưa seasonal qua `linear_seasonal`; cộng hai kết quả để tạo đầu ra cuối cùng.
- Chạy khối kiểm thử để quan sát lần lượt: Input, trend, seasonal, dự đoán từ nhánh trend, dự đoán từ nhánh seasonal và Output tổng hợp.

Mục tiêu học tập:

- Hiểu quy trình tách chuỗi bằng thao tác làm mượt rồi trừ: trước hết lấy trend bằng trung bình trượt, sau đó trừ trend khỏi tín hiệu gốc để có phần còn lại.
- Nắm cách hai nhánh tuyến tính xử lý riêng hai phần và được cộng lại để tạo dự báo, qua đó thấy được tính tách biệt và khả năng diễn giải của DLinear.

Bước 4: Xây dựng mô hình NLinear với trọng số cố định

Code Exercise

```

1 class NLinear(nn.Module):
2     def __init__(self):
3         super().__init__()
4         self.linear = nn.Linear(5, 3)
5         # Set weights cụ thể
6         with torch.no_grad():
7             self.linear.weight.data = torch.tensor([
8                 [0.1, 0.2, 0.1, 0.3, 0.3],
9                 [0.2, 0.1, 0.3, 0.2, 0.2],
10                [0.3, 0.2, 0.2, 0.2, 0.1]
11            ])
12            self.linear.bias.data = torch.tensor([0.5, 1.0, 2.0])
13
14    def forward(self, x):
15        # Normalize bằng last value
16        last_value = ***Your Code Here***
17        x_norm = ***Your Code Here***
18        pred_norm = ***Your Code Here***
19        pred = ***Your Code Here***
20        return pred
21
22 # Test NLinear
23 nlinear_model = NLinear()
24 nlinear_out = nlinear_model(data)
25
26 print("NLINEAR MODEL:")
27 last_val = data[:, -1:]
28 x_norm = data - last_val
29 pred_norm = nlinear_model.linear(x_norm)
30
31 print(f"    Input: {data.squeeze().numpy()}")
32 print(f"    Last value: {last_val.squeeze().item()}")
33 print(f"    Normalized: {x_norm.squeeze().numpy()}")
34 print(f"    Pred normalized: {pred_norm.squeeze().detach().numpy()}")
35 print(f"    Final output: {nlinear_out.squeeze().detach().numpy()}")
36
37 # Manual verification
38 print("    Manual calc:")
39 for i in range(3):
40     w = nlinear_model.linear.weight.data[i]
41     b = nlinear_model.linear.bias.data[i]

```

```

42 norm_pred = torch.sum(w * x_norm.squeeze()) + b
43 final = norm_pred + last_val.squeeze()
44 print(f"    Out{i+1}: {norm_pred:.3f} + {last_val.squeeze().item():.1f} = {final:.3f}")

```

Thực hành điền code vào các vị trí *****Your Code Here***** để hoàn thành các yêu cầu sau:

- Điền last_value là cột cuối của cửa sổ đầu vào theo trục thời gian.
- Chuẩn hoá đầu vào bằng cách trừ last_value khỏi toàn bộ dãy để thu được x_norm.
- Tính pred_norm bằng cách đưa x_norm qua lớp tuyến tính đã khai báo.
- Cộng lại mốc last_value vào pred_norm để có pred và trả về pred.

Mục tiêu học tập:

- Hiểu cơ chế tái căn tâm theo giá trị cuối cùng: mô hình học trên độ lệch so với mốc hiện tại, sau đó cộng mốc để quay về thang gốc.
- Quan sát chuỗi bước in ra: giá trị cuối, đầu vào đã chuẩn hoá, dự đoán trên không gian chuẩn hoá, và đầu ra cuối cùng.

VII. Bài toán dự đoán giá cổ phiếu sử dụng LTSF-Linear Models

Trong phần này, ta áp dụng họ mô hình **LTSF-Linear** (Linear, NLinear, DLinear) cho bài toán **dự đoán giá đóng cửa cổ phiếu VIC trong 7 ngày tới**. Mỗi mô hình được huấn luyện với các độ dài cửa sổ quá khứ khác nhau — 7, 30, 120 và 480 ngày — nhằm đánh giá khả năng học và khái quát trên nhiều quy mô thời gian. Kết quả dự đoán sẽ được so sánh với nhau để kiểm chứng hiệu quả và độ ổn định của các phương pháp tuyến tính trong dự báo chuỗi tài chính.

Bước 1: Import thư viện và tải dữ liệu

```

1 !gdown 18J_Z8b-qMMj9wm5eGyQ-1nPS16PfRePK
2
3 import pandas as pd
4 import numpy as np
5 import matplotlib.pyplot as plt
6 from matplotlib.ticker import PercentFormatter
7 import matplotlib.dates as mdates
8
9 import seaborn as sns
10 import warnings
11 warnings.filterwarnings("ignore")

```

```

12
13 import torch
14 import torch.nn as nn
15 from torch.utils.data import Dataset, DataLoader
16 from sklearn.metrics import mean_squared_error,
17 mean_absolute_error, r2_score
18 from sklearn.preprocessing import StandardScaler
19
20 plt.style.use("seaborn-v0_8")
21 sns.set_palette("husl")
22 print(f"Device available: {'CUDA' if torch.cuda.is_available() else 'CPU'}")
23
24 # Đọc dữ liệu
25 df = pd.read_csv("VIC.csv")
26 print(f"Dataset shape: {df.shape}")
27 print(f>Date range: {df["time"].min()} to {df["time"].max()}")
28
29 # Xử lý và sắp xếp theo thời gian
30 df["time"] = pd.to_datetime(df["time"])
31 df = df.sort_values("time").reset_index(drop=True)
32
33 # Thông tin cơ bản
34 print("\nFirst few rows:")
35 display(df.head())
36
37 print("\nData types:")
38 print(df.dtypes)
39
40 print("\nBasic statistics:")
41 display(df.describe())

```

Trong bước đầu tiên, các thư viện cần thiết được import, bao gồm: `pandas`, `numpy`, `matplotlib`, `seaborn` cho xử lý và trực quan hóa dữ liệu; `torch` và `sklearn` cho xây dựng mô hình và đánh giá hiệu suất. Tập dữ liệu **VIC.csv** được tải về và đọc bằng `pandas.read_csv()`, sau đó cột `time` được chuyển sang kiểu `datetime` và sắp xếp theo thứ tự thời gian.

Cuối cùng, dữ liệu được kiểm tra sơ bộ qua kích thước tập, khoảng thời gian, kiểu dữ liệu và thống kê mô tả cơ bản — giúp xác nhận cấu trúc dữ liệu đầu vào trước khi xử lý và huấn luyện mô hình.

	time	open	high	low	close	volume	symbol
0	2020-08-03	77.33	78.31	75.56	77.87	164310	VIC
1	2020-08-04	78.84	78.84	77.69	78.22	229230	VIC
2	2020-08-05	78.22	78.67	75.38	77.33	434490	VIC
3	2020-08-06	78.22	78.40	77.42	77.78	332340	VIC
4	2020-08-07	77.87	78.40	77.60	77.78	182500	VIC

Hình 14: Vài samples của tập dữ liệu VIC

Bước 2: Khám phá và tiền xử lý dữ liệu

```

1 # Tạo essential features
2 print("1. Tạo essential features:")
3 df["daily_return"] = df["close"].pct_change()
4 df["close_log"] = np.log(df["close"])
5 print(" - daily_return: phần trăm thay đổi hàng ngày")
6 print(" - close_log: log giá đóng cửa")
7
8 # Kiểm tra missing values cho TẤT CẢ các cột
9 missing_info = df.isnull().sum()
10
11 # Chỉ xử lý giá trị khuyết cần thiết
12 if df["daily_return"].isnull().sum() > 0:
13     df["daily_return"].fillna(0, inplace=True)
14
15 # Kiểm tra lại sau khi xử lý
16 missing_after = df.isnull().sum()
17 missing_after

```

Trong bước này, hai đặc trưng cơ bản được tạo ra từ giá đóng cửa:

- **daily_return**: phần trăm thay đổi giá giữa hai ngày liên tiếp, phản ánh biến động ngắn hạn.
- **close_log**: logarit tự nhiên của giá đóng cửa, giúp giảm độ lệch và ổn định phân phối dữ liệu.

Sau đó, tập dữ liệu được kiểm tra để phát hiện giá trị khuyết (*missing values*) trên toàn bộ các cột. Nếu xuất hiện giá trị NaN ở cột **daily_return** (do phép tính phần trăm thay đổi tại dòng đầu tiên), ta thay thế bằng 0 để đảm bảo tính toàn vẹn dữ liệu trước khi đưa vào huấn luyện.

Việc kiểm tra lại sau khi xử lý giúp xác nhận rằng toàn bộ dữ liệu hiện đã hoàn chỉnh, sẵn sàng cho các bước tiền xử lý và tạo tập huấn luyện tiếp theo.

```

1 fig, axes = plt.subplots(2, 2, figsize=(16, 12))
2 fig.suptitle("VIC Stock Data Analysis - After Feature Creation", fontsize=16,
3              fontweight="bold")
4
5 # 1) Stock Price Over Time
6 axes[0, 0].plot(df["time"], df["close"], linewidth=1.5, color="blue", alpha=0.8)
7 axes[0, 0].set_title("VIC Stock Price Over Time", fontsize=12, fontweight="bold")
8 axes[0, 0].set_xlabel("Date")
9 axes[0, 0].set_ylabel("Price (VND)")
10 axes[0, 0].grid(True, alpha=0.3)
11 axes[0, 0].tick_params(axis="x", rotation=45)
12
13 # 2) Log-transformed Price
14 axes[0, 1].plot(df["time"], df["close_log"], linewidth=1.5, color="green", alpha=0.8)
15 axes[0, 1].set_title("Log-transformed Stock Price", fontsize=12, fontweight="bold")

```

```

15 axes[0, 1].set_xlabel("Date")
16 axes[0, 1].set_ylabel("Log Price")
17 axes[0, 1].grid(True, alpha=0.3)
18 axes[0, 1].tick_params(axis="x", rotation=45)
19
20 # 3) Daily Returns Over Time
21 ret_ts = df[["time", "daily_return"]].dropna().copy()
22 ret_ts["ret_ma20"] = ret_ts["daily_return"].rolling(20).mean()
23
24 axes[1, 0].plot(ret_ts["time"], ret_ts["daily_return"], linewidth=1.2, alpha=0.85,
25                label="Daily Return")
26 axes[1, 0].plot(ret_ts["time"], ret_ts["ret_ma20"], linewidth=2.0, linestyle="--",
27                alpha=0.95, label="20D Avg")
28 axes[1, 0].axhline(0, color="black", linewidth=1, alpha=0.6)
29 axes[1, 0].set_title("Daily Returns Over Time", fontsize=12, fontweight="bold")
30 axes[1, 0].set_xlabel("Date")
31 axes[1, 0].set_ylabel("Return")
32 axes[1, 0].yaxis.set_major_formatter(PercentFormatter(1.0))
33 axes[1, 0].xaxis.set_major_locator(mdates.AutoDateLocator())
34 axes[1, 0].xaxis.set_major_formatter(mdates.ConciseDateFormatter(mdates.AutoDateLocator(
35                                     )))
36 axes[1, 0].tick_params(axis="x", rotation=0)
37 axes[1, 0].grid(True, alpha=0.3)
38 axes[1, 0].legend(fontsize=9)
39
40 # 4) Volume Over Time
41 axes[1, 1].plot(df["time"], df["volume"], linewidth=1.2, color="purple", alpha=0.75)
42 axes[1, 1].set_title("Trading Volume Over Time", fontsize=12, fontweight="bold")
43 axes[1, 1].set_xlabel("Date")
44 axes[1, 1].set_ylabel("Volume")
45 axes[1, 1].grid(True, alpha=0.3)
46 axes[1, 1].tick_params(axis="x", rotation=45)
47
48 plt.tight_layout()
49 plt.show()

```

Sau khi xử lý và bổ sung đặc trưng, dữ liệu được trực quan hoá để quan sát xu hướng và cấu trúc biến động theo thời gian:

- **(1) Stock Price Over Time:** biểu diễn giá đóng cửa theo thời gian, thể hiện biến động tổng thể của cổ phiếu VIC.
- **(2) Log-transformed Price:** đồ thị giá đã logarit hoá, giúp làm phẳng các dao động lớn, dễ quan sát xu hướng dài hạn.
- **(3) Daily Returns Over Time:** hiển thị phần trăm thay đổi giá hằng ngày cùng đường trung bình trượt 20 ngày, giúp nhận diện giai đoạn biến động mạnh hay ổn định.
- **(4) Trading Volume Over Time:** biểu đồ khối lượng giao dịch, phản ánh mức độ thanh khoản và sự quan tâm của thị trường.

Những biểu đồ này cung cấp cái nhìn trực quan đầu tiên về hành vi của cổ phiếu VIC — bao gồm xu hướng giá, độ biến động và mối liên hệ giữa giá và khối lượng giao dịch — là nền tảng quan trọng trước khi trích xuất đặc trưng đầu vào cho mô hình dự báo.



Hình 15: Các biểu đồ cung cấp cái nhìn trực quan về hành vi của cổ phiếu VIC

Bước 3: Chuẩn bị dữ liệu chuỗi thời gian đơn biến cho mô hình dự báo

```

1 class UnivariateTimeSeriesDataset(Dataset):
2     """Dataset for univariate time series forecasting - 7 days ahead"""
3     def __init__(self, data, seq_len, pred_len=7, target_col="close", normalize=False):
4         self.data = data.dropna().reset_index(drop=True)
5         self.seq_len = seq_len
6         self.pred_len = pred_len
7         self.target_col = target_col
8         self.normalize = normalize
9
10        self.series = self.data[target_col].values
11

```

```

12         if normalize:
13             self.mean = np.mean(self.series)
14             self.std = np.std(self.series)
15
16     def __len__(self):
17         return len(self.series) - self.seq_len - self.pred_len + 1
18
19     def __getitem__(self, idx):
20         x = self.series[idx:idx+self.seq_len].copy()
21         y = self.series[idx+self.seq_len:idx+self.seq_len+self.pred_len].copy()
22
23         if self.normalize:
24             x = (x - self.mean) / self.std
25             y = (y - self.mean) / self.std
26
27         return torch.FloatTensor(x), torch.FloatTensor(y)
28
29     def denormalize(self, normalized_values):
30         if not self.normalize:
31             return normalized_values
32         return normalized_values * self.std + self.mean
33
34
35 def create_univariate_datasets(df, seq_lengths, pred_len=7, target_col="close"):
36     """Create univariate datasets for different sequence lengths"""
37     datasets = {}
38
39     for seq_len in seq_lengths:
40         dataset = UnivariateTimeSeriesDataset(
41             data=df, seq_len=seq_len, pred_len=pred_len,
42             target_col=target_col, normalize=False
43         )
44         datasets[f"{seq_len}d"] = dataset
45
46     return datasets
47
48
49 # Create datasets for 7-day prediction with new input lengths
50 seq_lengths = [7, 30, 120, 480] # Input sequence lengths
51 pred_len = 7 # Predict 7 days ahead
52
53 # Create datasets using log prices for better stability
54 datasets = create_univariate_datasets(df, seq_lengths, pred_len, "close_log")
55
56 print(f"Created datasets for 7-day prediction with input lengths: {seq_lengths}")
57 for name, dataset in datasets.items():
58     print(f"- {name}: {len(dataset)} samples")
59
60 print("Using log-transformed prices for stability.")

```

Lớp `UnivariateTimeSeriesDataset` được định nghĩa để tạo tập dữ liệu dự báo đơn biến. Mỗi mẫu gồm hai phần: $\mathbf{x} \in \mathbb{R}^L$ (chuỗi đầu vào gồm L ngày trước) và $\mathbf{y} \in \mathbb{R}^T$ (giá trị mục tiêu cho T

ngày tiếp theo). Ở đây, dữ liệu sử dụng giá đóng cửa logarit (`close_log`) để tăng tính ổn định khi huấn luyện mô hình.

Các tập dữ liệu được tạo cho bốn độ dài của số đầu vào: **7, 30, 120 và 480** ngày, với mục tiêu dự báo **7 ngày tiếp theo**.

```

1 def create_time_based_splits(dataset, train_ratio=0.7, val_ratio=0.15):
2     """Create time-based splits for time series data"""
3     total_len = len(dataset)
4     train_len = int(total_len * train_ratio)
5     val_len = int(total_len * val_ratio)
6
7     # Time-based splits (no shuffling to preserve temporal order)
8     train_indices = list(range(0, train_len))
9     val_indices = list(range(train_len, train_len + val_len))
10    test_indices = list(range(train_len + val_len, total_len))
11
12    train_dataset = torch.utils.data.Subset(dataset, train_indices)
13    val_dataset = torch.utils.data.Subset(dataset, val_indices)
14    test_dataset = torch.utils.data.Subset(dataset, test_indices)
15
16    return train_dataset, val_dataset, test_dataset
17
18
19 # Create splits for all sequence lengths
20 data_splits = {}
21
22 print("Creating time-based data splits:")
23 print("- Train: 70%, Validation: 15%, Test: 15%")
24 print("- Temporal order preserved (no shuffling)")
25
26 for seq_name, dataset in datasets.items():
27     train, val, test = create_time_based_splits(dataset)
28     data_splits[seq_name] = {
29         "train": train,
30         "val": val,
31         "test": test
32     }
33
34 print(f"\nTotal datasets with splits: {len(data_splits)}")
35
36 # Verify split integrity
37 print(f"\n=== SPLIT VERIFICATION ===")
38 for seq_name, splits in data_splits.items():
39     total_samples = len(splits["train"]) + len(splits["val"]) + len(splits["test"])
40     original_samples = len(datasets[seq_name])
41
42     print(f"{seq_name}: {total_samples}/{original_samples} samples"
43           if total_samples == original_samples
44           else f"{seq_name}: ERROR - {total_samples}/{original_samples}")
45
46 print("\nData splits created successfully!")

```

Sau khi tạo tập dữ liệu, dữ liệu được chia theo tỷ lệ thời gian cố định: **70%** cho huấn luyện, **15%** cho xác thực và **15%** cho kiểm thử. Không áp dụng xáo trộn (*no shuffling*) để giữ nguyên thứ tự thời gian. Cách chia này đảm bảo tính liên tục của chuỗi và phản ánh đúng bối cảnh dự báo thực tế, đồng thời giúp đánh giá khả năng mô hình tổng quát hoá qua các giai đoạn khác nhau.

```

Creating time-based data splits:
- Train: 70%, Validation: 15%, Test: 15%
- Temporal order preserved (no shuffling)

Total datasets with splits: 4

=== SPLIT VERIFICATION ===
7d: 1487/1487 samples ✓
30d: 1464/1464 samples ✓
120d: 1374/1374 samples ✓
480d: 1014/1014 samples ✓

Data splits created successfully!

```

Hình 16: Tổng quan dữ liệu sau khi được chia thành các tập

Bước 4: Xây dựng và khởi tạo các mô hình LTSF-Linear

Code Exercise

```

1 # LTSF-Linear Models Implementation for Univariate Time Series (7-day prediction)
2 class Linear(nn.Module):
3     """Simple Linear model for univariate time series forecasting"""
4     def __init__(self, seq_len, pred_len=7):
5         super(Linear, self).__init__()
6         self.seq_len = seq_len
7         self.pred_len = pred_len
8         self.linear = ***Your Code Here***
9
10    def forward(self, x):
11        # x: [batch_size, seq_len] - historical prices
12        # Output: [batch_size, pred_len] - 7-day future predictions
13        return ***Your Code Here***
14
15    class DLinear(nn.Module):
16        """Decomposition Linear for univariate time series - handles trend and seasonality
17            """
18        def __init__(self, seq_len, pred_len=7, moving_avg=5):
19            super(DLinear, self).__init__()
20            self.seq_len = seq_len
21            self.pred_len = pred_len
22            self.moving_avg = min(moving_avg, seq_len - 1)
23
24            # Linear layers for trend and seasonal components
25            self.linear_trend = ***Your Code Here***
26            self.linear_seasonal = ***Your Code Here***
27
28            # Create moving average kernel for trend extraction

```

```

28         self.register_buffer('avg_kernel', torch.ones(1, 1, self.moving_avg) / self.
29                               moving_avg)
30
31     def decompose(self, x):
32         """Decompose series into trend and seasonal components"""
33         batch_size, seq_len = x.shape
34         x_reshaped = x.unsqueeze(1)
35
36         # Apply moving average for trend
37         padding = self.moving_avg // 2
38         x_padded = torch.nn.functional.pad(x_reshaped, (padding, padding), mode='
39                                     replicate')
40
41         trend = torch.nn.functional.conv1d(x_padded, self.avg_kernel, padding=0)
42         trend = trend.squeeze(1)
43
44         # Ensure trend has same length as input
45         if trend.shape[1] != seq_len:
46             trend = torch.nn.functional.interpolate(
47                 trend.unsqueeze(1), size=seq_len, mode='linear', align_corners=False
48             ).squeeze(1)
49
50         seasonal = ***Your Code Here***
51         return ***Your Code Here***
52
53     def forward(self, x):
54         trend, seasonal = ***Your Code Here***
55         trend_pred = ***Your Code Here***
56         seasonal_pred = ***Your Code Here***
57         return ***Your Code Here***
58
59 class NLinear(nn.Module):
60     """Normalized Linear for univariate time series - handles distribution shift"""
61     def __init__(self, seq_len, pred_len=7):
62         super(NLinear, self).__init__()
63         self.seq_len = seq_len
64         self.pred_len = pred_len
65         self.linear = ***Your Code Here***
66
67     def forward(self, x):
68         # Normalize by subtracting last value
69         last_value = ***Your Code Here***
70         x_normalized = ***Your Code Here***
71         pred_normalized = ***Your Code Here***
72         pred = ***Your Code Here***
73         return ***Your Code Here***
74
75 print("LTSF-Linear models implemented for 7-day univariate forecasting.")

```

Thực hành điền code vào các vị trí *****Your Code Here***** để hoàn thành các yêu cầu sau:

- Linear: khởi tạo lớp $[seq_len] \rightarrow [pred_len]$; trong forward, đưa x qua lớp đó để nhận đầu ra $[batch, pred_len]$.

- DLinear: tạo hai lớp cho trend và seasonal; trong decompose, tính seasonal bằng $x - trend$ và trả về (trend, seasonal); trong forward, dự đoán từng nhánh rồi cộng.
- NLinear: lấy $last_value = x[:, -1 :]$, chuẩn hoá $x - last_value$, dự đoán trên không gian chuẩn hoá, cộng lại mốc để ra kết quả.

Mục tiêu học tập:

- Hiểu đường đi dữ liệu và kích thước đầu vào–đầu ra ở cả ba biến thể.
- Hiểu cơ chế tách–cộng của DLinear và tái căn tâm theo giá trị cuối của NLinear.
- Giữ nguyên kích thước batch và trục thời gian; tránh dùng `squeeze()` gây mất chiều.

Sau đó ta tạo cấu hình và khởi tạo mô hình:

```

1  # Model Configuration and Creation for 7-day Prediction
2  horizon_configs = {
3      '7d': {'seq_len': 7, 'pred_len': 7},
4      '30d': {'seq_len': 30, 'pred_len': 7},
5      '120d': {'seq_len': 120, 'pred_len': 7},
6      '480d': {'seq_len': 480, 'pred_len': 7}
7  }
8
9  # Create model instances for each combination
10 model_configs = {
11     'Linear': {},
12     'DLinear': {},
13     'NLinear': {}
14 }
15
16 print("Creating model instances for 7-day prediction:")
17 for horizon, config in horizon_configs.items():
18     seq_len = config['seq_len']
19     pred_len = config['pred_len']
20
21     # Create model instances
22     model_configs['Linear'][horizon] = {
23         'model': Linear(seq_len, pred_len),
24         'seq_len': seq_len,
25         'pred_len': pred_len
26     }
27
28     model_configs['DLinear'][horizon] = {
29         'model': DLinear(seq_len, pred_len),
30         'seq_len': seq_len,
31         'pred_len': pred_len
32     }
33
34     model_configs['NLinear'][horizon] = {
35         'model': NLinear(seq_len, pred_len),
36         'seq_len': seq_len,
37         'pred_len': pred_len

```

```

38     }
39
40     print(f"    {horizon}: seq_len={seq_len}, pred_len={pred_len}")

```

Đoạn mã trên định nghĩa bốn cấu hình độ dài cửa sổ đầu vào (7, 30, 120, 480 ngày) cùng chung đích dự báo 7 ngày, rồi khởi tạo ba mô hình Linear, DLinear và NLinear cho từng cấu hình và lưu vào `model_configs` để phục vụ huấn luyện và so sánh hiệu suất.

Bước 5: Huấn luyện và trực quan hoá trực loss

Đầu tiên ta tạo dataset chuẩn hoá dùng chung một scaler, chia tập theo thời gian và đánh giá trên thang giá gốc.

```

1  # Training Configuration with Data Normalization for 7-day Prediction
2  device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
3
4  # Create normalized dataset class
5  class NormalizedDataset(Dataset):
6      def __init__(self, original_dataset, scaler=None):
7          self.original_dataset = original_dataset
8
9          # Fit scaler if not provided
10         if scaler is None:
11             all_data = []
12             for i in range(len(original_dataset)):
13                 x, y = original_dataset[i]
14                 all_data.extend(x.numpy())
15                 all_data.extend(y.numpy())
16
17             self.scaler = StandardScaler()
18             self.scaler.fit(np.array(all_data).reshape(-1, 1))
19         else:
20             self.scaler = scaler
21
22     def __len__(self):
23         return len(self.original_dataset)
24
25     def __getitem__(self, idx):
26         x, y = self.original_dataset[idx]
27
28         # Normalize features and targets
29         x_norm = self.scaler.transform(x.numpy().reshape(-1, 1)).flatten()
30         y_norm = self.scaler.transform(y.numpy().reshape(-1, 1)).flatten()
31
32         return torch.FloatTensor(x_norm), torch.FloatTensor(y_norm)
33
34     def denormalize(self, normalized_values):
35         """Convert normalized values back to original scale"""

```

```

36         return self.scaler.inverse_transform(normalized_values.reshape(-1, 1)).flatten
37             ()
38     # Create normalized datasets
39     normalized_datasets = {}
40     scaler = None
41
42     for horizon, dataset in datasets.items():
43         normalized_dataset = NormalizedDataset(dataset, scaler)
44         if scaler is None:
45             scaler = normalized_dataset.scaler # Use same scaler for all
46         normalized_datasets[horizon] = normalized_dataset
47
48     # Create normalized splits
49     normalized_data_splits = {}
50     for horizon in datasets.keys():
51         total_len = len(normalized_datasets[horizon])
52         train_len = int(total_len * 0.7)
53         val_len = int(total_len * 0.15)
54
55         train_indices = list(range(0, train_len))
56         val_indices = list(range(train_len, train_len + val_len))
57         test_indices = list(range(train_len + val_len, total_len))
58
59         normalized_data_splits[horizon] = {
60             'train': torch.utils.data.Subset(normalized_datasets[horizon], train_indices),
61             'val': torch.utils.data.Subset(normalized_datasets[horizon], val_indices),
62             'test': torch.utils.data.Subset(normalized_datasets[horizon], test_indices)
63         }
64
65     # Updated evaluation function for normalized data
66     def evaluate_model_normalized(model, test_loader, scaler, device='cpu'):
67         """Evaluate model performance with denormalization"""
68         model.eval()
69         predictions = []
70         actuals = []
71
72         with torch.no_grad():
73             for batch_x, batch_y in test_loader:
74                 batch_x, batch_y = batch_x.to(device), batch_y.to(device)
75                 outputs = model(batch_x)
76
77                 # Denormalize predictions and actuals
78                 for i in range(outputs.shape[0]):
79                     pred_denorm = scaler.inverse_transform(outputs[i].cpu().numpy().reshape
80                                                         (-1, 1)).flatten()
81                     actual_denorm = scaler.inverse_transform(batch_y[i].cpu().numpy().
82                                                         reshape(-1, 1)).flatten()
83                     predictions.extend(pred_denorm)
84                     actuals.extend(actual_denorm)
85
86         predictions = np.array(predictions)
87         actuals = np.array(actuals)

```

```

87     # Calculate metrics on original scale
88     mse = mean_squared_error(actuals, predictions)
89     mae = mean_absolute_error(actuals, predictions)
90     rmse = np.sqrt(mse)
91
92     try:
93         r2 = r2_score(actuals, predictions)
94     except:
95         r2 = -999
96
97     return {
98         'mse': mse,
99         'mae': mae,
100        'rmse': rmse,
101        'r2': r2,
102        'predictions': predictions,
103        'actuals': actuals
104    }

```

Lớp `NormalizedDataset` chuẩn hoá cả đầu vào và mục tiêu bằng cùng một `StandardScaler` dùng chung cho mọi horizon; tạo các tập train/val/test theo thứ tự thời gian; hàm `evaluate_model_normalized` suy luận trên dữ liệu chuẩn hoá, rồi đảo chuẩn hoá dự đoán và nhận để tính các chỉ số (MSE, MAE, RMSE, R^2) trên thang giá gốc.

```

1  def train_model(model, train_loader, val_loader, num_epochs=50, lr=0.001, device="cpu")
      :
2      """Train one model and return epoch-wise train/val MSE (no printing)."""
3      criterion = nn.MSELoss()
4      optimizer = torch.optim.Adam(model.parameters(), lr=lr)
5      scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=20, gamma=0.5)
6
7      model.to(device)
8      train_losses, val_losses = [], []
9
10     for _ in range(num_epochs):
11         # Train
12         model.train()
13         epoch_train = 0.0
14         for batch_x, batch_y in train_loader:
15             batch_x, batch_y = batch_x.to(device), batch_y.to(device)
16             optimizer.zero_grad()
17             preds = model(batch_x)
18             loss = criterion(preds, batch_y)
19             loss.backward()
20             optimizer.step()
21             epoch_train += loss.item()
22         train_losses.append(epoch_train / max(1, len(train_loader)))
23
24         # Validate
25         model.eval()
26         epoch_val = 0.0

```

```

27     with torch.no_grad():
28         for batch_x, batch_y in val_loader:
29             batch_x, batch_y = batch_x.to(device), batch_y.to(device)
30             preds = model(batch_x)
31             epoch_val += criterion(preds, batch_y).item()
32     val_losses.append(epoch_val / max(1, len(val_loader)))
33
34     scheduler.step()
35
36     return model, train_losses, val_losses

```

Hàm `train_model` huấn luyện mô hình và trả về lịch sử `train_losses` / `val_losses` cho việc vẽ đồ thị, *không* in log theo từng epoch.

```

1  import matplotlib.pyplot as plt
2
3  def plot_loss_curves(loss_history, horizons):
4      """Plot train/val losses for all models across horizons."""
5      for hz in horizons:
6          plt.figure(figsize=(8, 4.5))
7          for model_name in ["Linear", "DLinear", "NLinear"]:
8              if hz in loss_history.get(model_name, {}):
9                  t1 = loss_history[model_name][hz]["train"]
10                 v1 = loss_history[model_name][hz]["val"]
11                 plt.plot(t1, label=f"{model_name} - Train")
12                 plt.plot(v1, linestyle="--", label=f"{model_name} - Val")
13             plt.title(f"Loss Curves ({hz} input 7-day forecast)")
14             plt.xlabel("Epoch")
15             plt.ylabel("MSE Loss")
16             plt.grid(True, alpha=0.3)
17             plt.legend()
18             plt.tight_layout()
19             plt.show()
20
21  batch_size = 32
22  num_epochs = 50
23  learning_rate = 0.001
24
25  results = {"Linear": {}, "DLinear": {}, "NLinear": {}}
26  trained_models = {"Linear": {}, "DLinear": {}, "NLinear": {}}
27  loss_history = {"Linear": {}, "DLinear": {}, "NLinear": {}}
28
29  for horizon in ["7d", "30d", "120d", "480d"]:
30      train_loader = DataLoader(
31          normalized_data_splits[horizon]["train"],
32          batch_size=batch_size, shuffle=True, drop_last=True
33      )
34      val_loader = DataLoader(
35          normalized_data_splits[horizon]["val"],
36          batch_size=batch_size, shuffle=False, drop_last=True

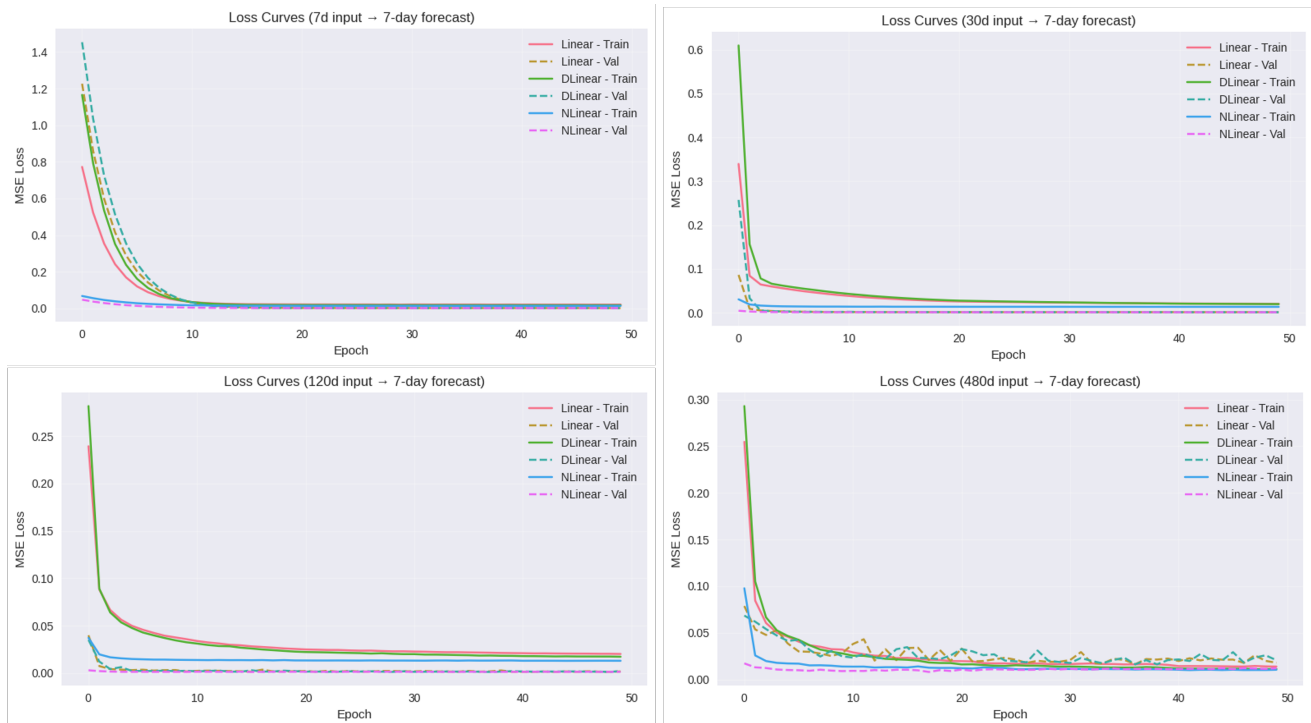
```

```

37 )
38 test_loader = DataLoader(
39     normalized_data_splits[horizon]["test"],
40     batch_size=batch_size, shuffle=False, drop_last=True
41 )
42
43 for model_name in ["Linear", "DLinear", "NLinear"]:
44     model = model_configs[model_name][horizon]["model"]
45     trained_model, tr_losses, va_losses = train_model(
46         model, train_loader, val_loader,
47         num_epochs=num_epochs, lr=learning_rate, device=device
48     )
49     loss_history[model_name][horizon] = {"train": tr_losses, "val": va_losses}
50     trained_models[model_name][horizon] = trained_model
51
52     # (Tuỳ chọn) lưu kết quả test để tổng hợp cuối cùng, không in
53     test_results = evaluate_model_normalized(trained_model, test_loader, scaler,
54                                             device)
55     results[model_name][horizon] = test_results
56
57 # Chỉ trực quan hoá (không in log)
58 plot_loss_curves(loss_history, horizons=["7d", "30d", "120d", "480d"])

```

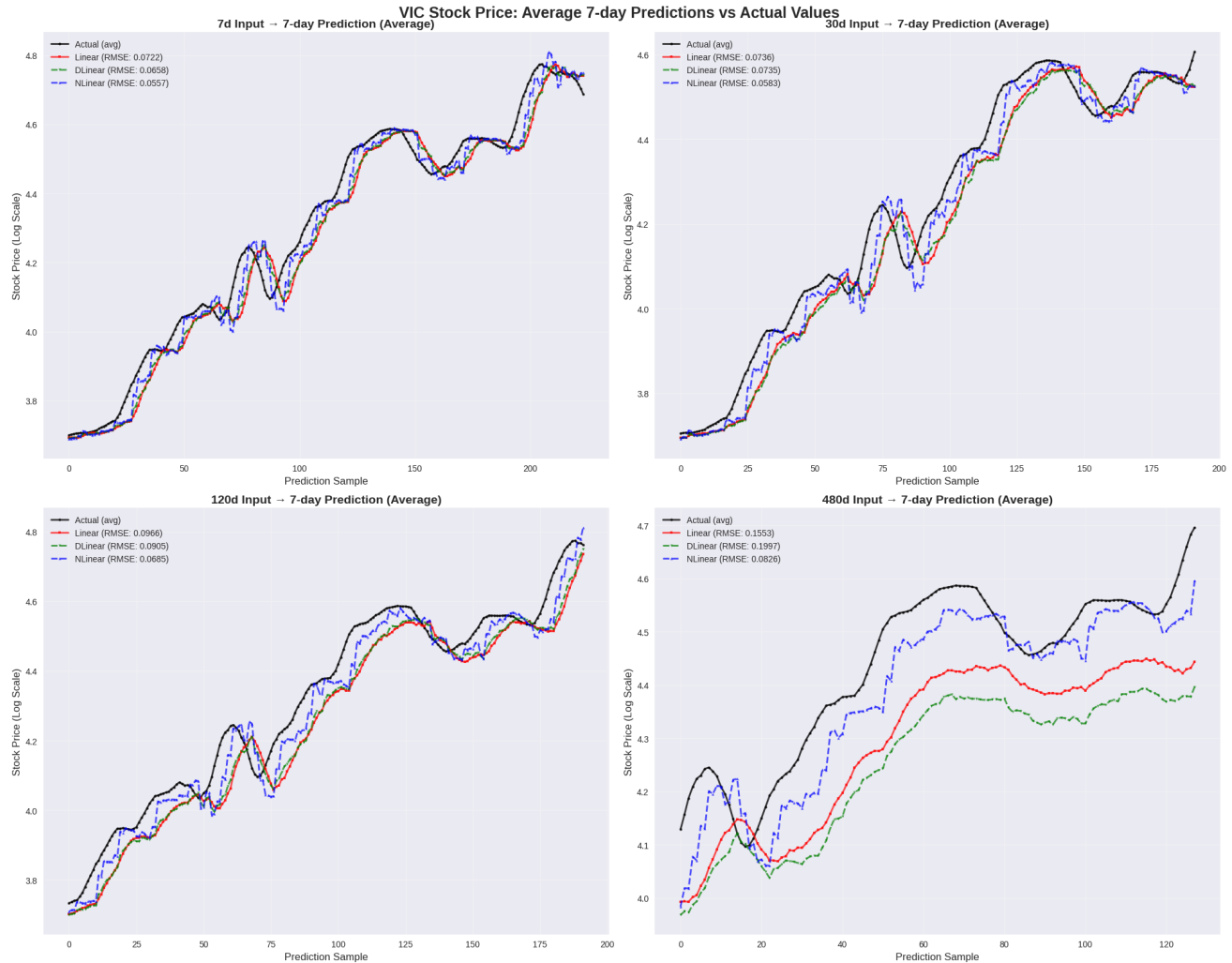
Khối lệnh trên huấn luyện toàn bộ các mô hình **Linear**, **DLinear**, **NLinear** trên bốn độ dài cửa sổ đầu vào và chỉ **vẽ** đường học (train/val) cho từng cấu hình, giúp theo dõi hội tụ mà không cần in log chi tiết.



Hình 17: Trực quan hóa loss của 4 độ dài input khác nhau

Sau khi quá trình huấn luyện hoàn tất, các mô hình Linear, DLinear và NLinear được sử dụng để dự đoán giá đóng cửa cổ phiếu VIC trong 7 ngày kế tiếp, dựa trên dữ liệu đầu vào có độ dài lần lượt là 7, 30, 120 và 480 ngày.

Các biểu đồ dưới đây trực quan hóa kết quả dự đoán, thể hiện sự so khớp giữa giá *thực tế* và *giá dự đoán* của từng mô hình. Mỗi subplot tương ứng với một độ dài cửa sổ đầu vào, giúp quan sát rõ mức độ ảnh hưởng của kích thước lịch sử đến độ chính xác của dự báo.



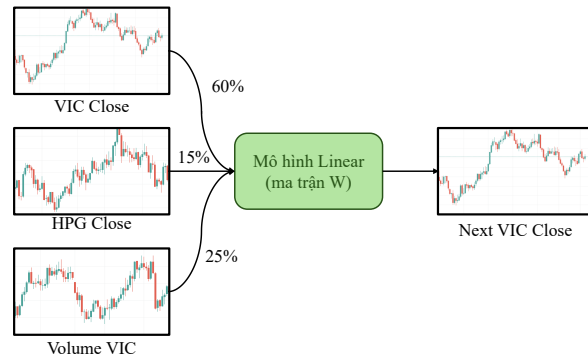
Hình 18: So sánh giữa giá thực tế và giá dự đoán từ các mô hình cho từng cửa sổ đầu vào.

VIII. Hướng cải tiến và mở rộng

Chúng ta đã thấy được sức mạnh của các mô hình LTSF-Linear. Với sự đơn giản, tốc độ và hiệu quả trong việc xử lý nhiều thông qua các cơ chế như chuẩn hóa (NLinear) hay phân rã (DLinear), chúng đóng vai trò là một baseline cực kỳ mạnh mẽ và dễ xử dụng. Tuy nhiên, chính sự đơn giản dựa trên nền tảng tuyến tính cũng là giới hạn của lớp mô hình này. Sau khi đã có một nền tảng vững chắc, đây là hai hướng cải tiến và mở rộng tự nhiên nhất mà bạn có thể khám phá.

VIII.1. Mở rộng sang bài toán đa biến

Hạn chế hiện tại: Toàn bộ project này đang tập trung vào bài toán đơn biến, tức là chỉ dùng lịch sử giá của chính cổ phiếu VIC để dự đoán tương lai của nó. Điều này bỏ qua một thực tế quan trọng trên thị trường tài chính: giá của một cổ phiếu hiếm khi di chuyển một mình. Nó chịu ảnh hưởng mạnh mẽ từ các yếu tố bên ngoài như chỉ số thị trường chung (VN-Index), giá của các cổ phiếu đầu ngành khác (ví dụ: HPG, FPT), hoặc thậm chí là các chỉ số kinh tế vĩ mô.

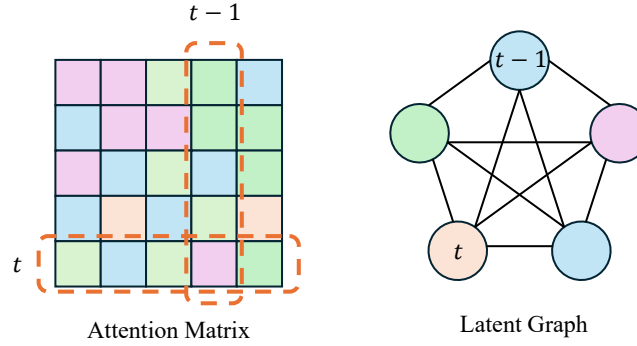


Hình 19: Với bài toán đa biến, mô hình sẽ quyết định được bao nhiêu phần trăm đóng góp mà mỗi giá trị đầu vào ảnh hưởng tới kết quả cuối cùng.

Hướng cải tiến: Hướng đi đầu tiên là **thu thập dữ liệu đa biến**, bổ sung thêm các chuỗi thời gian của các mã cổ phiếu liên quan, chỉ số VN-Index, hoặc các dữ liệu kinh tế khác. Tiếp theo, cần **điều chỉnh mô hình** bằng cách mở rộng các mô hình Linear, NLinear, và DLinear để chấp nhận đầu vào đa biến (ví dụ: input $\mathbf{x} \in \mathbb{R}^{L \times C}$ với C là số lượng biến), điều này đòi hỏi phải thay đổi kiến trúc của lớp `nn.Linear` và cách xử lý dữ liệu. Một bài toán mà chúng ta phải giải quyết là làm thế nào để áp dụng cơ chế phân rã của DLinear hay chuẩn hóa của NLinear một cách hiệu quả trên nhiều biến cùng lúc. Đây là một câu hỏi mở và là một hướng nghiên cứu thú vị.

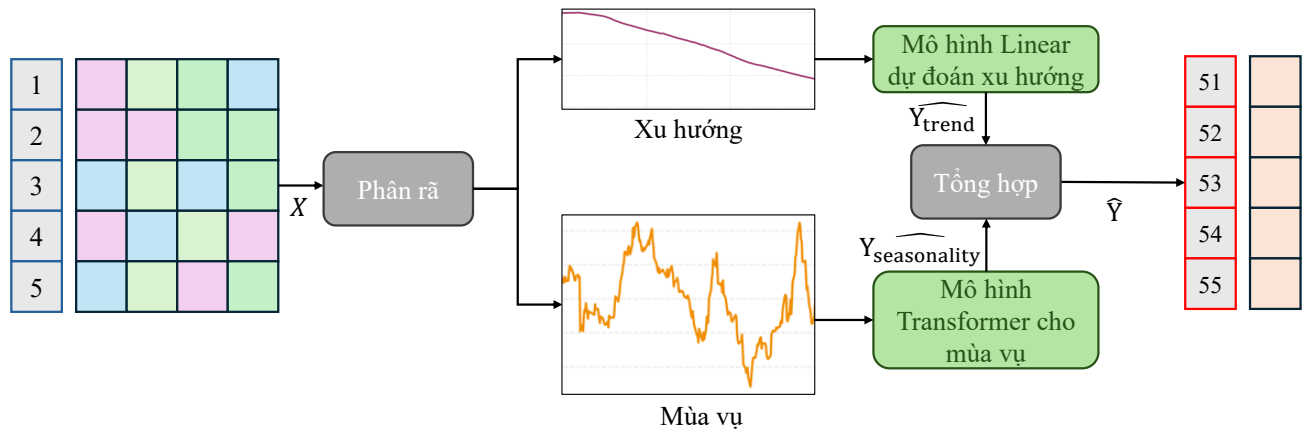
VIII.2. Hybrid model: kết hợp phân rã và transformer

Hạn chế hiện tại: Ở phần giới thiệu, chúng ta đã chỉ ra điểm yếu của Transformer là độ phức tạp tính toán và nguy cơ học vẹt nhiều. Ngược lại, điểm mạnh nhất của DLinear là khả năng tách tín hiệu thành xu hướng và mùa vụ. Vậy tại sao không kết hợp điểm mạnh của cả hai?



Hình 20: Với lớp mô hình Transformer, khi dự đoán tương lai chúng ta có thể hình lại toàn bộ quá khứ. Đồng thời, mô hình không cần chạy qua các giai đoạn để đến được tương lai.

Hướng cải tiến: Chúng ta có thể xây dựng một mô hình lai ghép (hybrid). Bước 1 là phân rã, sử dụng moving average từ DLinear để tách chuỗi đầu vào $\mathbf{x}$ thành hai thành phần: $\mathbf{x}_{trend}$ và $\mathbf{x}_{seasonal}$. Bước 2 là xử lý hai luồng dữ liệu của chúng ta có một cách riêng biệt. Với nhánh trend ($\mathbf{x}_{trend}$), thành phần này thường rất mượt và dễ đoán nên chúng ta chỉ cần dùng một mô hình **Linear** đơn giản để dự đoán nó. Ngược lại, nhánh seasonal ($\mathbf{x}_{seasonal}$) phức tạp hơn, chứa các dao động và mẫu khó nắm bắt; đây chính là nơi một mô hình **Transformer** (ví dụ: Autoformer hoặc một Transformer đơn giản) có thể phát huy thế mạnh để tìm ra các phụ thuộc phức tạp. Cuối cùng, bước 3 là tổng hợp, cộng kết quả dự đoán từ hai nhánh lại để ra được dự đoán cuối cùng.



Hình 21: Sơ đồ huấn luyện mô hình mẫu cho cách cải tiến hybrid nói trên. Chúng ta có thể sử dụng block Attention đơn giản hoặc toàn bộ mô hình phức tạp như Autoformer vào khu vực Transformer. Cách kết hợp vẫn giữ nguyên là phép cộng.

IX. Câu hỏi trắc nghiệm

- (Lý thuyết) Dự báo nhiều bước trực tiếp (Direct Multi-step, DMS). Phát biểu nào đúng về DMS trong các mô hình LTSF-Linear?
 - Dự báo tuần tự từng bước và dùng đầu ra bước t làm đầu vào bước $t+1$ (recursive/rollout).
 - Dự báo đồng thời T bước tương lai thông qua một ánh xạ duy nhất $\mathbb{R}^L \rightarrow \mathbb{R}^T$.
 - Cần RNN/Transformer để dự báo được T bước.
 - Luôn yêu cầu đổi tần suất (resampling) của chuỗi trước khi học.
- (Lý thuyết) Trong DLinear, $\text{MA}_m(\cdot)$ được dùng với mục đích chính là:
 - Ước lượng thành phần tần số thấp (low-pass) của chuỗi để thu được trend.
 - Khuếch đại thành phần tần số cao (high-pass) nhằm nhấn mạnh seasonal/residual.
 - Thay thế dữ liệu gốc bằng z-score.
 - Sinh nhân phân lớp hai lớp.
- (Lý thuyết) Với cửa sổ $\mathbf{x} \in \mathbb{R}^L$, chọn mốc $\ell = x_L$, đặt $\mathbf{x}' = \mathbf{x} - \ell \mathbf{1}_L$, dự báo $\hat{\mathbf{y}}' = W\mathbf{x}' + b$, hoàn nguyên $\hat{\mathbf{y}} = \hat{\mathbf{y}}' + \ell \mathbf{1}_T$. Tính chất dịch mức cộng tính (additive level-shift equivariance) nào sau đây là đúng?
 - $\hat{\mathbf{y}}(\mathbf{x} + \Delta \mathbf{1}_L) = \hat{\mathbf{y}}(\mathbf{x})$.
 - $\hat{\mathbf{y}}(\mathbf{x} + \Delta \mathbf{1}_L) = \hat{\mathbf{y}}(\mathbf{x}) + \Delta \mathbf{1}_T$.
 - $\hat{\mathbf{y}}(\mathbf{x} + \Delta \mathbf{1}_L) = 2\hat{\mathbf{y}}(\mathbf{x})$.
 - Không có bảo toàn nào liên quan đến dịch mức.
- (Tính toán) MA với phép lặp biên (nghĩa lặp lại các giá trị ở biên của chuỗi dữ liệu để đệm cho các phép tính ở đầu và cuối chuỗi). Cho $m=3$, $\mathbf{x} = [10, 12, 11, 13]$. Tính $\text{MA}_3(\mathbf{x})$ theo

$$\text{MA}_3(x)_t = \frac{1}{3}(x_{t-1}^* + x_t^* + x_{t+1}^*), \quad x^* \text{ lặp biên (replicate)}.$$

- [10.67, 11.00, 12.00, 12.33]
 - [11.00, 11.00, 12.00, 12.00]
 - [10.00, 11.00, 11.67, 13.00]
 - [10.67, 11.33, 11.67, 12.33]
- (Tính toán) NLinear (re-centering). $L=3$, $T=2$. $\mathbf{x} = [2, 3, 5]$, $\ell = x_3 = 5$. $W = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}$, $b = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$. Tính $\hat{\mathbf{y}}$:

$$\mathbf{x}' = \mathbf{x} - \ell \mathbf{1}, \quad \hat{\mathbf{y}}' = W\mathbf{x}' + b, \quad \hat{\mathbf{y}} = \hat{\mathbf{y}}' + \ell \mathbf{1}.$$
 - [2, 2]

- (b) [3, 3]
 (c) [4, 4]
 (d) [6, 6]
6. (Tính toán) NLinear – minh hoạ tính chất shift. Cho $L = 2$, $T = 1$, $\mathbf{x} = [2, 5]$, mốc $\ell = x_2 = 5$, $W = [1, 0]$, $b = 0$. Tính $\hat{y}(\mathbf{x})$ và $\hat{y}(\mathbf{x}+3\mathbf{1})$. Kết quả là:
- (a) $\hat{y}(\mathbf{x}) = 2$, $\hat{y}(\mathbf{x}+3\mathbf{1}) = 2$
 (b) $\hat{y}(\mathbf{x}) = 2$, $\hat{y}(\mathbf{x}+3\mathbf{1}) = 5$
 (c) $\hat{y}(\mathbf{x}) = 2$, $\hat{y}(\mathbf{x}+3\mathbf{1}) = -1$
 (d) $\hat{y}(\mathbf{x}) = 2$, $\hat{y}(\mathbf{x}+3\mathbf{1}) = 4$
7. (Tính toán) DLinear – phân rã và hai đầu tuyến tính. $L=4$, $T=1$, $m=3$, lặp biên. $\mathbf{x} = [10, 12, 11, 13]$, $\mathbf{x}_t = \text{MA}_3(\mathbf{x})$, $\mathbf{x}_s = \mathbf{x} - \mathbf{x}_t$. $W_t = \begin{bmatrix} \frac{1}{2} & \frac{1}{2} & 0 & 0 \end{bmatrix}$, $b_t = 0$; $W_s = \begin{bmatrix} 1 & 0 & 0 & 0 \end{bmatrix}$, $b_s = 1$. Tính $\hat{y} = (W_t \mathbf{x}_t + b_t) + (W_s \mathbf{x}_s + b_s)$.
- (a) 11.16
 (b) 12.05
 (c) 12.52
 (d) 13.09
8. (Tính toán) MSE (DMS) cho một mẫu. $T=3$, $\hat{\mathbf{y}} = [3, 4, 5]$, $\mathbf{y} = [2, 5, 5]$. Tính $\frac{1}{3} \sum_{t=1}^3 (\hat{y}_t - y_t)^2$.
- (a) 0.2222
 (b) 0.3333
 (c) 0.6667
 (d) 1.0000
9. (Tính toán) RMSE cho 7 bước. Với $\mathbf{e} = [1, -2, 2, -1, 0, 3, -3]$, $\text{RMSE} = \sqrt{\frac{1}{7} \sum_{t=1}^7 e_t^2}$ bằng:
- (a) 1.732
 (b) 2.000
 (c) 2.160
 (d) 2.236
10. (Tính toán) Linear – ánh xạ theo trục thời gian. $L=4$, $T=2$, $\mathbf{x} = [1, 0, 2, 1]$. $W = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}$, $b = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$. Tính $\hat{\mathbf{y}} = W\mathbf{x} + b$.
- (a) [3, 2]
 (b) [2, 3]
 (c) [1, 4]
 (d) [4, 1]

Phụ lục

- Hint:** Các file code gợi ý và dữ liệu nếu có được lưu trong thư mục có thể được tải [tại đây](#).
- Solution:** Các file code cài đặt hoàn chỉnh và phần trả lời nội dung trắc nghiệm có thể được tải [tại đây](#) (**Lưu ý:** Sáng thứ 3 khi hết deadline phần project, ad mới copy các nội dung bài giải nêu trên vào đường dẫn).
- Q&A:** Bạn có thể đặt thêm câu hỏi về nội dung bài đọc trong group Facebook hỏi đáp tại [đây](#). Tất cả câu hỏi sẽ được trả lời trong vòng tối đa 4 giờ.

AIO_QAs-Verified

🔒 Nhóm Riêng tư · 1,4K thành viên



Hình 22: Hình ảnh group facebook AIO Q&A

- Đề xuất phương pháp cải tiến project:** Một số phương pháp đề xuất cải tiến model này xem thêm ở VIII.
- Rubric:**

LTSF-Linear Project - Rubric		
Phần	Kiến Thức	Đánh Giá
1	- Nắm vững các khái niệm cơ bản của dự báo chuỗi thời gian. - Hiểu các đặc tính của dữ liệu tài chính (nhiều, phi tuyến tính, không dừng) và các thành phần của chuỗi (xu hướng, mùa vụ, nhiễu).	- Phân tích tại sao một mô hình đơn giản lại có thể hoạt động tốt trong miền dữ liệu nhiễu. - Trả lời các câu hỏi trắc nghiệm lý thuyết liên quan đến khái niệm DMS và các thách thức của LTSF.
2	- Hiểu kiến trúc cơ sở của mô hình Linear. - Nắm rõ cơ chế phân rã của DLinear. - Nắm rõ cơ chế chuẩn hóa của NLinear.	- Hoàn thiện code (from scratch) cho các hàm 'forward' của ba mô hình Linear, DLinear, và NLinear. - Hoàn thiện code định nghĩa các lớp mô hình Linear, DLinear, NLinear trong project thực tế
3	- Hiểu quy trình xử lý dữ liệu chuỗi thời gian thực tế. - Nắm bắt các hướng cải tiến và mở rộng: mở rộng sang bài toán đa biến (multivariate) hoặc kết hợp mô hình lai (hybrid) như DLinear + Transformer.	- Xây dựng pipeline dữ liệu cho bài toán dự đoán giá cổ phiếu VIC. - Phân tích kết quả dự đoán và thảo luận về giới hạn cũng như các hướng cải tiến tiếp theo.

LTSF-Linear Forecasting Challenge

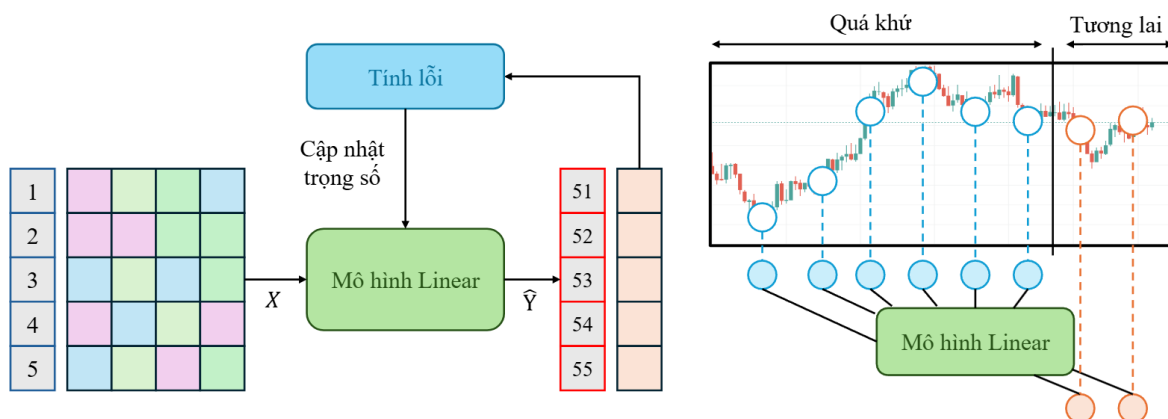
Nguyễn Quốc Thái

Hồ Quang Hiển

Đình Quang Vinh

I. Giới thiệu

LTSF-Linear Forecasting Challenge tập trung vào việc đánh giá hiệu quả của các mô hình tuyến tính trong bài toán dự báo chuỗi thời gian dài hạn. Thử thách ưu tiên việc khám phá sức mạnh của các mô hình tuyến tính như Linear, NLinear và DLinear khi áp dụng vào dữ liệu tài chính thực tế, thay vì dựa vào các mô hình deep learning phức tạp.



Hình 1: Mô phỏng quy trình chạy và huấn luyện mô hình Linear.

Mục tiêu của thử thách là xây dựng mô hình dự báo giá cổ phiếu FPT cho 100 ngày tương lai, dựa trên một cửa sổ quan sát đầu vào gồm N ngày. Bài toán sử dụng cơ chế dự báo lặp, trong đó mô hình chỉ dự đoán M ngày ở mỗi bước; các giá trị dự đoán này tiếp tục được dùng làm đầu vào cho bước dự báo kế tiếp. Cách tiếp cận này mô phỏng sát hơn kịch bản dự báo dài hạn trong thực tế, nơi sai số được tích lũy theo thời gian và mô hình cần giữ được độ ổn định qua nhiều vòng dự đoán.

Thử thách khuyến khích người tham gia:

- Thử nghiệm nhiều cách biến đổi dữ liệu để chuẩn hóa đầu vào.
- So sánh các biến thể mô hình tuyến tính khác nhau.

- Điều chỉnh linh hoạt kích thước cửa sổ đầu vào và kích thước đầu ra.

Các mô hình gợi ý bao gồm:

- **Linear cơ bản:** đóng vai trò baseline, giúp đánh giá mức cải thiện của các mô hình tuyến tính nâng cao.
- **NLinear:** chuẩn hóa và tái biến đổi dữ liệu, cải thiện độ ổn định và khả năng hội tụ.
- **DLinear:** tiếp cận theo hướng decomposition, tách dữ liệu thành thành phần xu hướng và mùa vụ trước khi mô hình hóa.

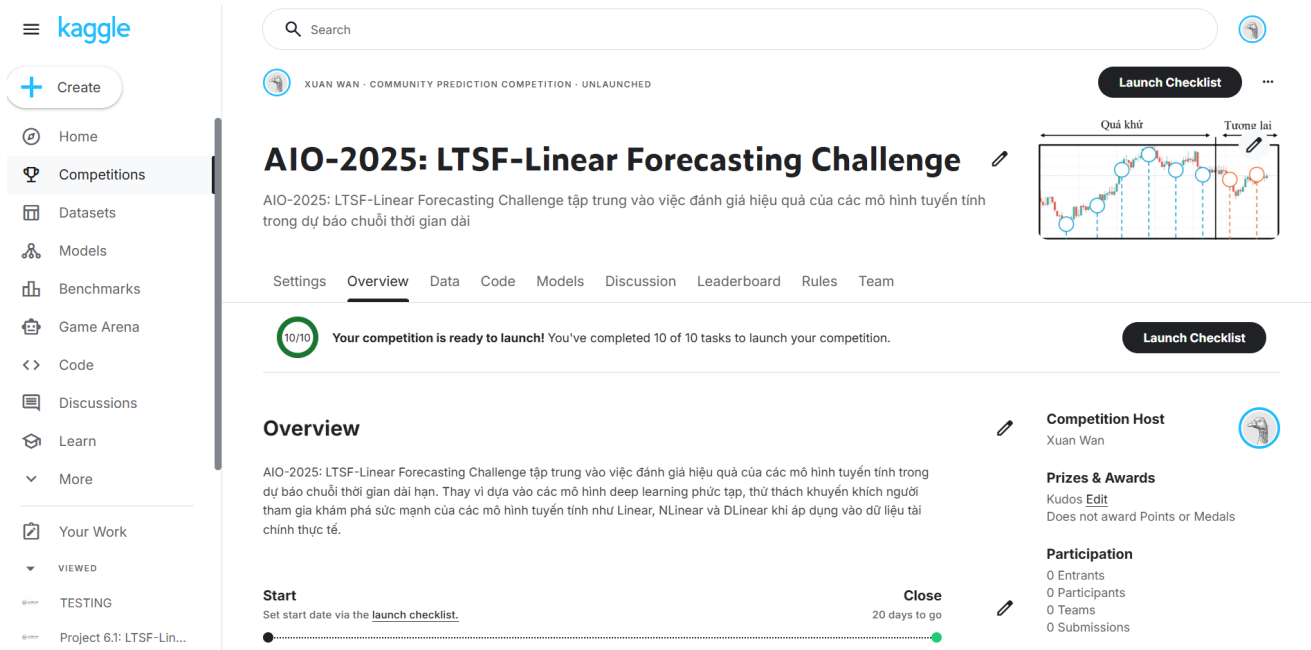
Mục tiêu cuối cùng không chỉ là đạt điểm số dự báo cao, mà còn:

- Xây dựng một pipeline dự báo rõ ràng, có cấu trúc, có thể tái lập kết quả.
- Phân tích và lý giải mô hình, thể hiện hiểu biết về đặc trưng của dữ liệu chuỗi thời gian tài chính.
- Góp phần thúc đẩy cộng đồng nghiên cứu và ứng dụng các mô hình tuyến tính trong dự báo chuỗi thời gian dài hạn.

Mục lục

I.	Giới thiệu	1
II.	Tổng quan về Competition	4
II.1.	Data	4
II.2.	Yêu cầu	5
II.3.	Giới hạn	6
II.4.	Đánh giá	6
II.5.	Submission	7
II.6.	Timeline và link truy cập cuộc thi	7
III.	Xây dựng baseline mẫu cho bài toán	8
IV.	Hướng dẫn nộp bài kết quả trên Kaggle Competition	15
IV.1.	Chạy baseline trên Kaggle và cải tiến code	15
IV.2.	Chạy notebook và nộp bài	17
IV.3.	Nộp file submission và xem kết quả	19
	Phụ lục	22

II. Tổng quan về Competition



Hình 2: Giao diện cuộc thi trên Kaggle

Cuộc thi được tổ chức trên nền tảng Kaggle với mục tiêu tạo ra một môi trường thực hành dự báo chuỗi thời gian bằng các mô hình tuyến tính. Người tham gia sẽ sử dụng dữ liệu lịch sử giá cổ phiếu để xây dựng mô hình dự báo 100 ngày tương lai, đồng thời tìm kiếm các cách biến đổi và khai thác dữ liệu nhằm cải thiện hiệu năng. Competition được thiết kế theo chuẩn Kaggle, bao gồm các tập dữ liệu, cơ chế chấm điểm bằng MSE, cùng hai leaderboard public và private giúp đánh giá mô hình một cách khách quan và toàn diện.

Truy cập cuộc thi trên Kaggle theo đường link [TẠI ĐÂY](#)

II.1. Data

Bộ dữ liệu sử dụng là lịch sử giao dịch cổ phiếu FPT với tần suất theo ngày. Mỗi dòng tương ứng với một phiên giao dịch và chứa các thông tin giá và khối lượng tiêu chuẩn trong phân tích tài chính. Bộ dữ liệu được dùng để xây dựng và huấn luyện các mô hình dự báo chuỗi thời gian.

Các tập dữ liệu được cung cấp bao gồm:

- **FPT_train.csv**: tập dữ liệu train.
- **sample_submission.csv**: file submission mẫu, đúng theo định dạng yêu cầu.

Các cột trong dataset train **FPT_train.csv**:

- **time**: ngày diễn ra phiên giao dịch, định dạng YYYY-MM-DD.
- **open**: giá mở cửa của cổ phiếu trong ngày.
- **high**: mức giá cao nhất đạt được trong phiên.
- **low**: mức giá thấp nhất trong phiên.
- **close**: giá đóng cửa, thường được sử dụng làm biến mục tiêu trong các mô hình dự báo.
- **volume**: khối lượng cổ phiếu được giao dịch trong phiên.
- **symbol**: mã chứng khoán.

Các cột trong file nộp bài mẫu `sample_submission.csv`:

- **id**: chỉ số của từng điểm dự báo trong tập test, từ 1 đến 100.
- **close**: giá đóng cửa được mô hình dự đoán cho từng id. Trong file mẫu, các giá trị được biểu diễn dạng $x_1, x_2, \dots, x_{100}$ để minh họa vị trí mà thí sinh phải điền kết quả dự báo thật.

II.2. Yêu cầu

Xây dựng mô hình dự báo giá đóng cửa cổ phiếu FPT cho **100 ngày trong tương lai**, dựa trên dữ liệu lịch sử đã được cung cấp trong tập train. Khuyến khích phân tích sâu đặc trưng chuỗi thời gian và tìm các phương pháp biến đổi đầu vào nhằm giúp mô hình tuyến tính hoạt động hiệu quả hơn.



Hình 3: Minh họa ý tưởng mô hình NLinear.

Tập trung triển khai và so sánh các mô hình **tuyến tính** như Linear, NLinear, DLinear hoặc các biến thể tuyến tính khác. Bên cạnh việc đánh giá ưu nhược điểm của từng mô hình, thí sinh có thể đề xuất các ý tưởng cải tiến hoặc tinh chỉnh kiến trúc để phù hợp hơn với đặc thù của bài toán dự báo.

Xây dựng một **pipeline hoàn chỉnh** bao gồm: tiền xử lý dữ liệu, huấn luyện mô hình, dự báo trên tập test và tạo tập submission. Pipeline cần rõ ràng, có khả năng chạy lại và đảm bảo tính tái lập kết quả.

Tạo tập **submission.csv** đúng chuẩn, bao gồm đầy đủ các giá trị dự báo theo yêu cầu của bài toán và tuân thủ định dạng đã được mô tả trong phần hướng dẫn.

II.3. Giới hạn

Thử thách tập trung vào các mô hình thuần tuyến tính; do đó, các mô hình deep learning phức tạp không phải là trọng tâm và không được khuyến khích sử dụng trong phạm vi cuộc thi. Người tham gia được yêu cầu xây dựng và tối ưu các mô hình tuyến tính dựa trên dữ liệu đã cung cấp, hướng tới việc khai thác đặc trưng chuỗi thời gian một cách hiệu quả và phù hợp với bản chất của các mô hình này.

Bên cạnh đó, việc huấn luyện chỉ được phép sử dụng dữ liệu FPT đã cho trong tập train. Không được sử dụng thêm dữ liệu bên ngoài, dữ liệu bổ sung hoặc dữ liệu trong tương lai. Những giới hạn này nhằm đảm bảo tính công bằng giữa các thí sinh và giúp tập trung vào khả năng phân tích, xử lý và khai thác tập dữ liệu cho trước.

II.4. Đánh giá

Metric được sử dụng trong cuộc thi là **MSE (Mean Squared Error)**. MSE đo lường sai lệch giữa giá trị dự đoán và giá trị thực tế bằng cách lấy trung bình bình phương sai số. Giá trị MSE **càng thấp** cho thấy mô hình dự báo càng chính xác và ổn định.

Trong quá trình chấm điểm, tập benchmark được chia thành hai phần: **public test** và **private test**. Điểm số hiển thị trên leaderboard trong suốt thời gian diễn ra cuộc thi được tính dựa trên **public test**. Sau khi cuộc thi kết thúc, hệ thống sẽ sử dụng **private test** để tính lại điểm cuối cùng và xác định thứ hạng chính thức. Cách phân chia này giúp hạn chế việc mô hình bị tối ưu hóa theo leaderboard và đảm bảo đánh giá khách quan hơn.

AIO-2025: LTSF-Linear Forecasting Challenge
Launch Checklist
...

[Settings](#)
[Overview](#)
[Data](#)
[Code](#)
[Models](#)
[Discussion](#)
[Leaderboard](#)
[Rules](#)
[Team](#)

[All](#)
[Your Work](#)
[Shared With You](#)
[Bookmarks](#)
Hotness ▼

Pinned notebooks



SIMPLE BASELINE (BTC-AIVN)
Updated 3d ago
Score: 422.4133 · 0 comments · AIO-2025: LTSF-Linear Forecasting Challenge

▲
3

Hình 4: Kết quả đánh giá của Baseline cuộc thi

II.5. Submission

Notebook của người thi phải tạo ra file **submission.csv** để Kaggle lấy file này để chấm điểm.

File bao gồm 2 cột:

- **id**: chỉ số của từng mẫu dự báo trong tập test, từ 1 đến 100.
- **close**: giá cổ phiếu FPT được mô hình dự báo cho từng id tương ứng.

File cần chứa đúng 100 dòng dự báo, mỗi dòng ứng với một giá trị close cho từng id trong khoảng 1–100.

Benchmark			Submission.csv		
1	id, Usage, close		1	id, close	
2	1, Public, 118.55		2	1, 123.25592	
3	2, Public, 117.17		3	2, 129.32845	
4	3, Public, 117.69		4	3, 129.05875	
5	4, Public, 113.29		5	4, 120.17197	
6	5, Public, 112.09		6	5, 130.66904	
7	6, Public, 112.09		7	6, 130.212	
8	7, Public, 107.43		8	7, 121.64471	
9	8, Public, 107.78		9	8, 130.4356	
10	9, Public, 110.36		10	9, 129.36853	

Hình 5: Định dạng mẫu của file submission.csv so với benchmark của BTC.

II.6. Timeline và link truy cập cuộc thi

- **End Date**: 12/06/2025, 23:59 PM
- **Private Leaderboard Release**: 12/06/2025, 23:59 PM
- **Timezone**: Asia/Ho_Chi_Minh (GMT +07:00)
- **Link Kaggle Competition**: [Truy cập cuộc thi tại đây](#)

III. Xây dựng baseline mẫu cho bài toán

Baseline được xây dựng với mục tiêu cung cấp một ví dụ đơn giản, rõ ràng cho bài toán dự báo chuỗi thời gian dài hạn. Ta bắt đầu bằng việc chuẩn bị công cụ và môi trường lập trình.

Bước 1: Import thư viện cần thiết

Đây là bước khởi tạo baseline, bao gồm tải các thư viện xử lý dữ liệu, tính toán số học và xây dựng mô hình học sâu.

```
1 import pandas as pd
2 import numpy as np
3 import torch
4 import torch.nn as nn
5 from torch.utils.data import Dataset, DataLoader
6 import matplotlib.pyplot as plt
```

Các thư viện trên đảm nhiệm vai trò:

- **pandas, numpy**: xử lý dữ liệu dạng bảng và mảng số.
- **torch**: xây dựng mô hình và thực hiện tính toán tensor.
- **torch.nn**: định nghĩa các lớp mạng tuyến tính, hàm mất mát, tối ưu.
- **Dataset, DataLoader**: tổ chức dữ liệu huấn luyện.
- **matplotlib**: trực quan hóa kết quả.

Bước 2: Chuẩn bị dữ liệu và tạo đặc trưng

Sau khi đã import các thư viện cần thiết, bước tiếp theo trong baseline là tải dữ liệu huấn luyện, sắp xếp theo thời gian và tạo thêm đặc trưng `close_log` giúp mô hình ổn định hơn khi học trên dữ liệu tài chính.

```
1 # ===== THAM SỐ CẤU HÌNH =====
2 INPUT_LEN = 14          # Số ngày input
3 OUTPUT_LEN = 3          # Số ngày output mỗi lần dự đoán
4 TOTAL_PREDICT_DAYS = 100 # Tổng số ngày cần dự đoán
5
6 # Load data train
7 df_train = pd.read_csv("/kaggle/input/aio-2025-linear-forecasting-challenge/FPT_train.csv")
8 df_train["time"] = pd.to_datetime(df_train["time"])
9 df_train = df_train.sort_values("time").reset_index(drop=True)
10
11 # Tạo log transform
```

```

12 df_train["close_log"] = np.log(df_train["close"])
13
14 print(f"Train data shape: {df_train.shape}")
15 print(f>Date range: {df_train["time"].min()} to {df_train["time"].max()}")
16
17 print(df_train[["time", "close", "close_log"]].head())

```

Đến đây, dữ liệu đã sẵn sàng cho việc tạo các mẫu (window) huấn luyện trong bước kế tiếp. Việc thêm biến `close_log` giúp giảm dao động do biên độ giá lớn và thường mang lại kết quả tốt hơn đối với các mô hình tuyến tính.

Bước 3: Phân tích dữ liệu và trực quan hóa

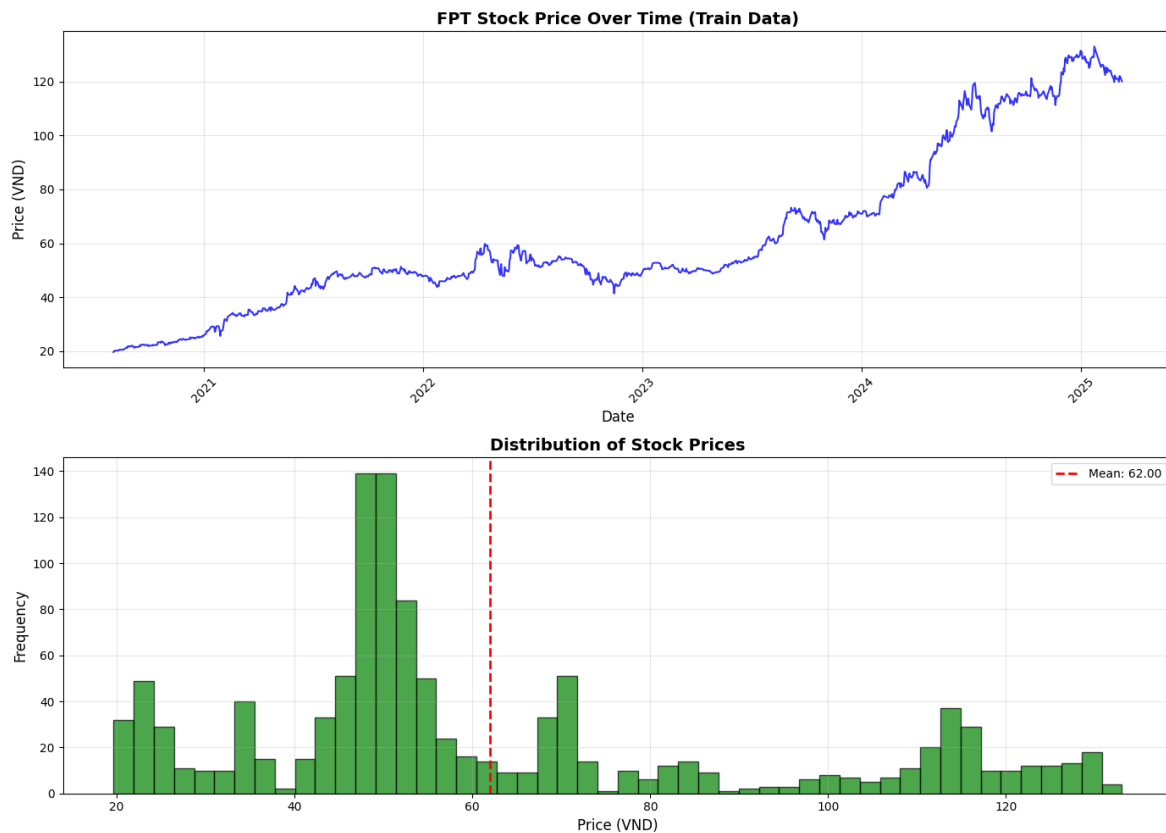
Ở bước này, baseline thực hiện trực quan hóa dữ liệu để quan sát xu hướng giá và phân phối giá cổ phiếu FPT. Điều này giúp hiểu đặc trưng chuỗi thời gian trước khi xây dựng mô hình.

```

1  # ===== PHÂN TÍCH DỮ LIỆU =====
2  # Visualization
3  fig, axes = plt.subplots(2, 1, figsize=(14, 10))
4
5  # Plot 1: Giá close theo thời gian
6  axes[0].plot(df_train["time"], df_train["close"], linewidth=1.5, color="blue", alpha=0.8)
7  axes[0].set_title("FPT Stock Price Over Time (Train Data)", fontsize=14, fontweight="bold")
8  axes[0].set_xlabel("Date", fontsize=12)
9  axes[0].set_ylabel("Price (VND)", fontsize=12)
10 axes[0].grid(True, alpha=0.3)
11 axes[0].tick_params(axis="x", rotation=45)
12
13 # Plot 2: Phân phối giá
14 axes[1].hist(df_train["close"], bins=50, color="green", alpha=0.7, edgecolor="black")
15 axes[1].set_title("Distribution of Stock Prices", fontsize=14, fontweight="bold")
16 axes[1].set_xlabel("Price (VND)", fontsize=12)
17 axes[1].set_ylabel("Frequency", fontsize=12)
18 axes[1].grid(True, alpha=0.3)
19 axes[1].axvline(
20     df_train["close"].mean(),
21     color="red",
22     linestyle="--",
23     linewidth=2,
24     label=f"Mean: {df_train['close'].mean():.2f}"
25 )
26 axes[1].legend()
27
28 plt.tight_layout()
29 plt.show()

```

Hai biểu đồ cho phép quan sát rõ xu hướng biến động giá theo thời gian và sự phân bố giá, giúp đánh giá mức độ ổn định, biên độ dao động và tính mùa vụ của chuỗi thời gian.



Hình 6: Biểu đồ phân tích dữ liệu cổ phiếu

Bước 4: Xây dựng mô hình Linear

Tiếp theo, baseline định nghĩa mô hình Linear đơn giản và tạo dataset theo dạng cửa sổ trượt để mô hình có thể học quan hệ giữa chuỗi đầu vào và chuỗi dự báo.

```

1 # Linear Model
2 class LinearModel(nn.Module):
3     def __init__(self, input_len, output_len):
4         super(LinearModel, self).__init__()
5         self.input_len = input_len
6         self.output_len = output_len
7         self.linear = nn.Linear(input_len, output_len)
8
9     def forward(self, x):
10        # x: [batch_size, input_len]
11        # output: [batch_size, output_len]
12        return self.linear(x)
13

```

```

14 model = LinearModel(INPUT_LEN, OUTPUT_LEN)
15
16 # Dataset
17 class TimeSeriesDataset(Dataset):
18     def __init__(self, data, input_len, output_len, target_col="close_log"):
19         self.data = data[target_col].values
20         self.input_len = input_len
21         self.output_len = output_len
22
23     def __len__(self):
24         return len(self.data) - self.input_len - self.output_len + 1
25
26     def __getitem__(self, idx):
27         x = self.data[idx : idx + self.input_len]
28         y = self.data[idx + self.input_len : idx + self.input_len + self.output_len]
29         return torch.FloatTensor(x), torch.FloatTensor(y)
30
31 # Tạo dataset và dataloader
32 dataset = TimeSeriesDataset(df_train, INPUT_LEN, OUTPUT_LEN)
33 train_loader = DataLoader(dataset, batch_size=32, shuffle=True)
34
35 print(f"Dataset size: {len(dataset)} samples")
36 print(f"Sample shape: input={INPUT_LEN}, output={OUTPUT_LEN}")

```

Dataset dạng sliding window cho phép mô hình học dựa trên các đoạn chuỗi liên tiếp. Mỗi mẫu gồm `input_len` điểm quá khứ làm đầu vào và `output_len` điểm tương lai làm nhãn.

Bước 5: Huấn luyện mô hình Linear

Sau khi định nghĩa mô hình và dataset, baseline tiến hành huấn luyện mô hình Linear bằng hàm mất mát MSE và tối ưu Adam. Mỗi epoch mô hình học cách dự đoán chuỗi tương lai dựa trên các cửa sổ dữ liệu quá khứ.

```

1 # Training
2 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
3 model = model.to(device)
4
5 criterion = nn.MSELoss()
6 optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
7
8 num_epochs = 30
9 train_losses = []
10
11 for epoch in range(num_epochs):
12     model.train()
13     epoch_loss = 0.0
14
15     for batch_x, batch_y in train_loader:
16         batch_x, batch_y = batch_x.to(device), batch_y.to(device)

```

```

17         optimizer.zero_grad()
18         outputs = model(batch_x)
19         loss = criterion(outputs, batch_y)
20         loss.backward()
21         optimizer.step()
22
23         epoch_loss += loss.item()
24
25     avg_loss = epoch_loss / len(train_loader)
26     train_losses.append(avg_loss)
27
28     if (epoch + 1) % 5 == 0:
29         print(f"Epoch [{epoch+1}/{num_epochs}], Loss: {avg_loss:.6f}")
30
31 print("Training completed!")
32

```

Sau quá trình huấn luyện, mô hình đã học được mối quan hệ tuyến tính giữa 14 ngày quá khứ và 3 ngày tương lai, sẵn sàng cho bước suy luận dự báo 100 ngày theo cơ chế recursive.

Bước 6: Dự báo tương lai bằng recursive forecasting

Mô hình sẽ dự báo lặp từng bước: từ 14 ngày quá khứ dự báo 3 ngày tương lai, sau đó ghép kết quả vào cuối chuỗi để dự báo tiếp, cho đến khi đủ 100 ngày. Toàn bộ dự báo được thực hiện trên không gian log và sau đó chuyển ngược về giá thật.

```

1  # Gán input inference ban đầu (14 ngày cuối từ train data)
2  initial_input = df_train["close_log"].values[-INPUT_LEN:].copy()
3  current_input = initial_input.copy()
4
5  # Dự đoán iterative: 14 -> 3 -> 3 -> ... cho đến 100 ngày
6  model.eval()
7  all_predictions_log = []
8
9  # Bắt đầu với input ban đầu
10 current_input = initial_input.copy()
11
12 with torch.no_grad():
13     remaining_days = TOTAL_PREDICT_DAYS
14
15     while remaining_days > 0:
16         # Số ngày cần dự đoán trong lần này
17         predict_this_step = min(OUTPUT_LEN, remaining_days)
18
19         # Dự đoán
20         input_tensor = torch.FloatTensor(current_input).unsqueeze(0).to(device)
21         output = model(input_tensor)
22         predictions = output[0].cpu().numpy()[:predict_this_step]
23
24         # Lưu predictions

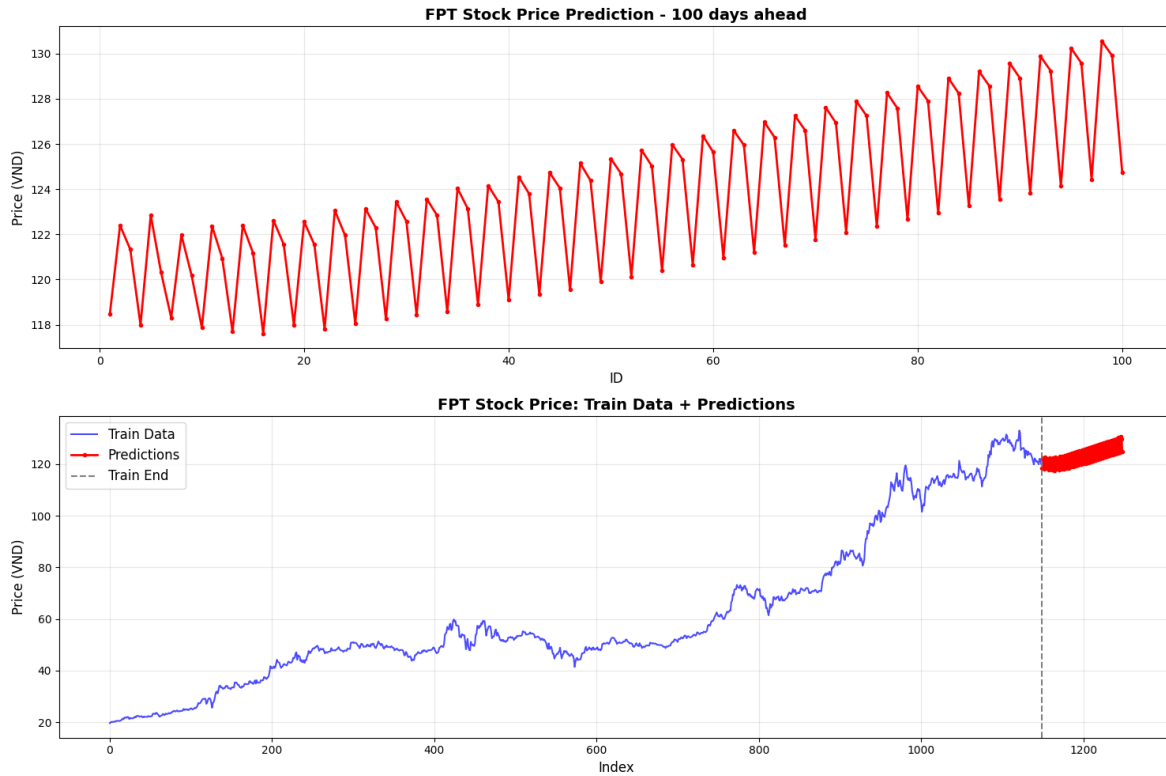
```

```

25     all_predictions_log.extend(predictions)
26
27     # Cập nhật input cho lần tiếp theo: bỏ INPUT_LEN - OUTPUT_LEN ngày đầu, thêm
        predictions
28     current_input = np.concatenate([
29         current_input[OUTPUT_LEN:], # Bỏ OUTPUT_LEN ngày đầu
30         predictions # Thêm predictions mới
31     ])
32
33     remaining_days -= predict_this_step
34
35     # Chuyển đổi từ log về giá close
36     predictions_close = np.exp(all_predictions_log)
37
38     # Id: 100 số nguyên liên tiếp từ 1 đến 100
39     prediction_ids = list(range(1, TOTAL_PREDICT_DAYS + 1))
40
41     # Visualization kết quả dự đoán
42     fig, axes = plt.subplots(2, 1, figsize=(15, 10))
43
44     # Plot 1: Dự đoán 100 ngày tiếp theo
45     axes[0].plot(prediction_ids, predictions_close, linewidth=2, color="red",
46                 marker="o", markersize=3)
47     axes[0].set_title(f"FPT Stock Price Prediction - {TOTAL_PREDICT_DAYS} days ahead",
48                      fontsize=14, fontweight="bold")
49     axes[0].set_xlabel("ID", fontsize=12)
50     axes[0].set_ylabel("Price (VND)", fontsize=12)
51     axes[0].grid(True, alpha=0.3)
52
53     # Plot 2: Kết hợp train data và predictions
54     last_train_idx = len(df_train)
55     axes[1].plot(range(len(df_train)), df_train["close"], linewidth=1.5, color="blue",
56                 alpha=0.7, label="Train Data")
57     axes[1].plot(range(last_train_idx, last_train_idx + TOTAL_PREDICT_DAYS),
58                 predictions_close, linewidth=2, color="red", marker="o",
59                 markersize=3, label="Predictions")
60     axes[1].axvline(last_train_idx, color="gray", linestyle="--", linewidth=1.5,
61                    label="Train End")
62     axes[1].set_title("FPT Stock Price: Train Data + Predictions",
63                      fontsize=14, fontweight="bold")
64     axes[1].set_xlabel("Index", fontsize=12)
65     axes[1].set_ylabel("Price (VND)", fontsize=12)
66     axes[1].legend(fontsize=12)
67     axes[1].grid(True, alpha=0.3)
68
69     plt.tight_layout()
70     plt.show()

```

Cơ chế recursive giúp mô phỏng đúng bài toán dự báo dài hạn, nơi sai số bị tích lũy sau mỗi bước và mô hình phải duy trì ổn định qua nhiều vòng dự đoán.



Hình 7: Baseline dự đoán giá của 100 ngày tiếp theo

Bước 7: Tạo file submission.csv đúng định dạng

Sau khi mô hình dự báo xong 100 giá trị tương lai, baseline tạo file `submission.csv` theo đúng format Kaggle: gồm hai cột `id` và `close`, với 100 dòng dự báo.

```

1 # Tạo file CSV output với format: id, close
2 output_df = pd.DataFrame({
3     "id": prediction_ids,
4     "close": predictions_close
5 })
6
7 # Lưu file CSV
8 output_filename = "submission.csv"
9 output_df.to_csv(output_filename, index=False)
10 print(f"Đã lưu file: {output_filename}")

```

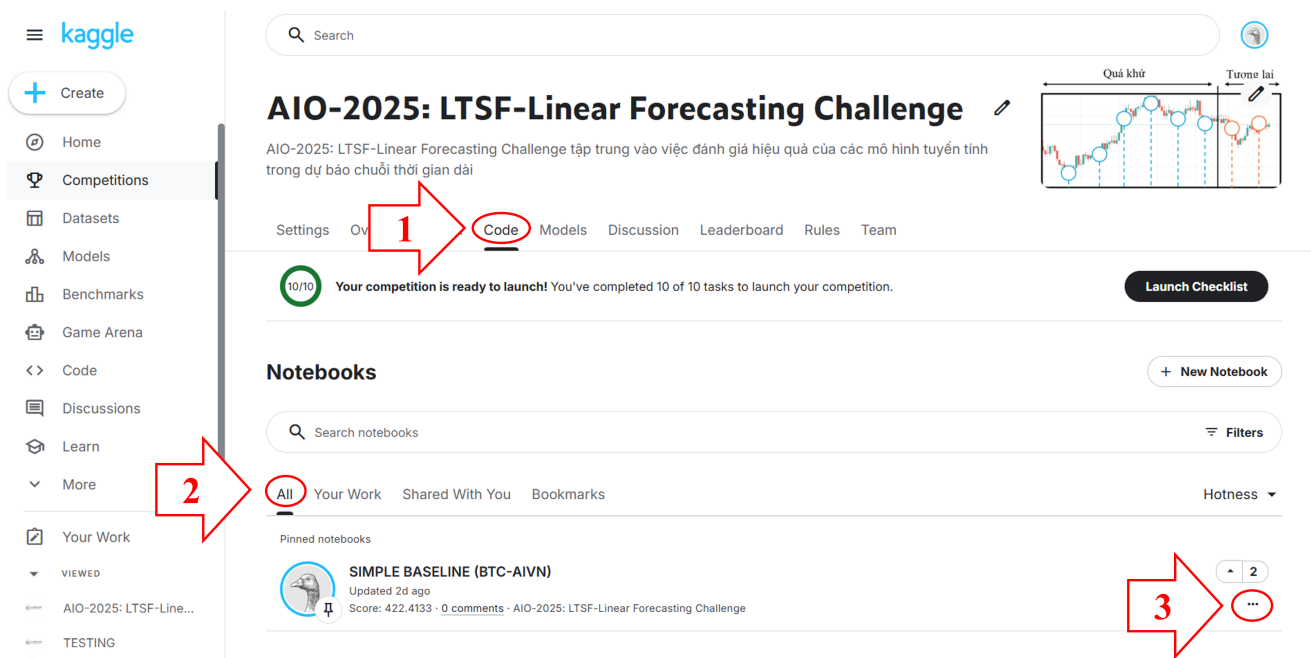
Kết quả là file `submission.csv` sẵn sàng để nộp lên hệ thống chấm điểm của Kaggle.

IV. Hướng dẫn nộp bài kết quả trên Kaggle Competition

Ở phần này chúng ta sẽ tìm hiểu quy trình nộp bài cho một cuộc thi trên Kaggle theo đúng thứ tự hệ thống chấm điểm của nền tảng. Người tham gia sẽ bắt đầu từ việc mở và sao chép notebook baseline, sau đó chỉnh sửa hoặc chạy notebook để sinh ra tệp submission.csv. Ngoài ra, còn có thể nộp bài trực tiếp thông qua notebook hoặc tải thủ công tệp submission lên trang cuộc thi.

Video hướng dẫn nộp bài trên Kaggle, các bạn có thể xem [TẠI ĐÂY](#)

IV.1. Chạy baseline trên Kaggle và cải tiến code



Hình 8: Chọn tab Code và notebook baseline.

Sau khi truy cập vào cuộc thi, chúng ta thực hiện:

- Nhấn vào tab **Code** trên trang competition (mũi tên 1).
- Ở phần Notebooks, chọn **All** để hiển thị tất cả notebook (mũi tên 2).
- Tại notebook baseline muốn dùng để nộp bài, nhấn nút ... ở bên phải để mở menu thao tác (mũi tên 3).

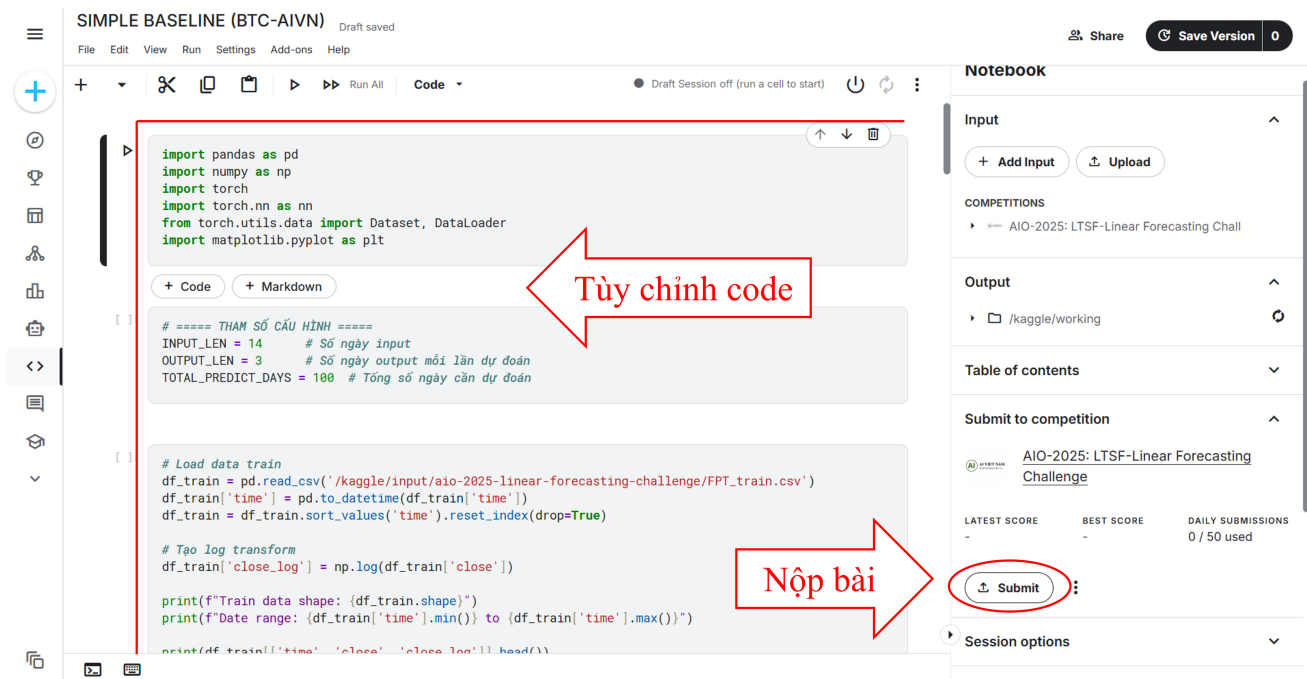
The screenshot displays the Kaggle interface for the AIO-2025: LTSF-Linear Forecasting Challenge. The left sidebar contains navigation links: Home, Competitions, Datasets, Models, Benchmarks, Game Arena, Code, Discussions, Learn, and More. The main content area shows the competition title, a description, and a progress indicator (10/10 tasks completed). Below this, the 'Notebooks' section is visible, featuring a search bar and a list of pinned notebooks. The notebook 'SIMPLE BASELINE (BTC-AIVN)' is highlighted, and its context menu is open, showing options like 'Copy and Edit in Kaggle Notebooks', 'Copy and Edit in Colab Notebooks', 'Download', 'Add to Collection', 'Bookmark', and 'Unpin'. A red arrow points to the 'Copy and Edit in Kaggle Notebooks' option, with the text 'Tùy chỉnh baseline' (Customize baseline) written next to it.

Hình 9: Copy và Edit baseline notebook.

Sau khi menu hiện ra như hình trên, chọn mục **Copy and Edit in Kaggle Notebooks** để mở notebook **SIMPLE BASELINE** trong môi trường Kaggle và chuẩn bị chạy notebook để sinh file **submission.csv**.

Nếu muốn tạo một notebook mới từ đầu thì nhấn vào ô "New Notebook"

IV.2. Chạy notebook và nộp bài



Hình 1: Tùy chỉnh code và nhấn Submit để nộp bài

Trong bước này, thực hiện:

- Tùy chỉnh (hoặc chạy thử) phần code trong notebook sao cho sinh ra file `submission.csv`, file được tạo nằm trong folder output và Kaggle sẽ lấy file để tính điểm.
- Ở panel **Submit to competition** bên phải nhấn nút **Submit** để gửi notebook lên hệ thống chấm điểm, hệ thống sẽ chạy lại tất cả cell và tính điểm tự động dựa vào file `submission.csv` được tạo.

The screenshot displays a Kaggle Notebook interface for a competition submission. The notebook is titled "SIMPLE BASELINE (BTC-AIVN)". The code cell shows the following imports and data loading:

```
import pandas as pd
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
import matplotlib.pyplot as plt
```

The output cell shows the submission status as "Complete". The right sidebar shows the "Submit to competition" section with a table of scores and submission counts. Red annotations highlight the "Chấm điểm xong" (Scoring completed) status and the "Kết quả" (Results) table.

LATEST SCORE	BEST SCORE	DAILY SUBMISSIONS
1922.4857 V1	1922.4857 V1	1 / 50 used

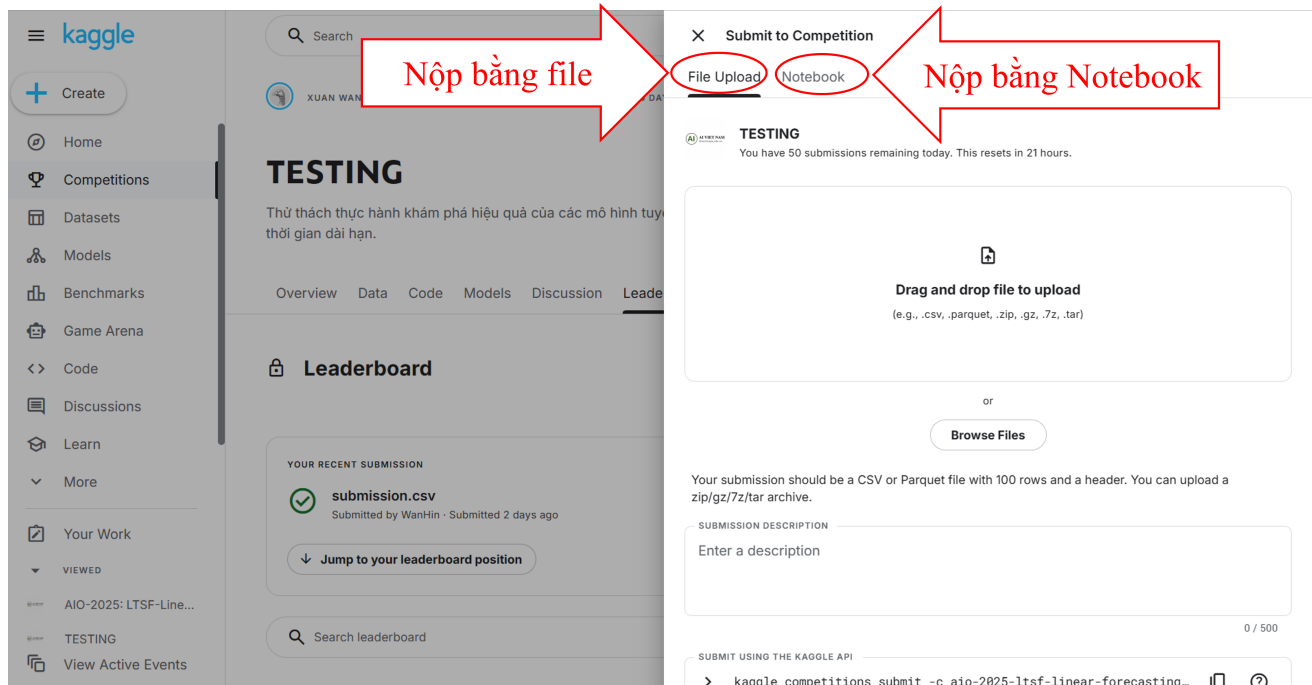
Hình 2: Notebook được chấm điểm và hiển thị kết quả

Sau khi Kaggle chạy xong notebook:

- Ở góc bên trái phía dưới xuất hiện mục **Competition submission** với trạng thái **Complete**.
- Trong phần **Submit to competition** sẽ hiện **Latest score**, **Best score** và số lần nộp trong ngày, cho biết bài nộp đã được chấm thành công.

IV.3. Nộp file submission và xem kết quả

Ngoài cách nộp bài ở Bước 2 ta vẫn còn cách khác để nộp bài.



Hình 3: Nhấn nút Submit Prediction trên tab Leaderboard

ta cần thực hiện các bước:

- Trên trang competition, chuyển sang tab **Leaderboard**.
- Nhấn nút **Submit Prediction** ở góc trên bên phải để mở cửa sổ nộp bài.

AIO-2025: LTSF-Linear Forecasting Challenge

Settings Overview Data Code Models Discussion **Leaderboard** Rules Team

YOUR RECENT SUBMISSION

✓ submission.csv
Submitted by WanHin · Submitted 18 minutes ago

Score: 1922.4857
Private score: 7817.7584

↓ Jump to your leaderboard position

Search leaderboard

Public Private

This leaderboard is calculated with approximately 50% of the test data. The final results will be based on the other 50%, so the final standings may be different.

#	Team	Members	Score	Entries	Last	Join
1	WanHin		1922.4857	1	18m	

🤖 Your First Entry!
Welcome to the leaderboard!

Hình 4: Chọn cách nộp: File Upload hoặc Notebook

Tại cửa sổ **Submit to Competition**:

- Chọn tab **File Upload** nếu đã có sẵn file `submission.csv` và muốn upload trực tiếp.
- Hoặc chọn tab **Notebook** nếu muốn nộp thông qua một notebook đã chạy trên Kaggle.
- Với cách nộp file, kéo thả hoặc chọn file `submission.csv` rồi xác nhận nộp bài.

TESTING

Thử thách thực hành khám phá hiệu quả của các mô hình tuyến tính như NLinear và DLinear trong dự báo chuỗi thời gian dài hạn.

Overview Data Code Models Discussion **Leaderboard** Rules Team Submissions

Leaderboard

Raw Data Refresh

YOUR RECENT SUBMISSION

✓ submission.csv
Submitted by WanHin · Submitted 2 days ago

Score: 350.76
Private score:

↓ Jump to your leaderboard position

Search leaderboard

Hình 5: Kết quả điểm số và thứ hạng trên leaderboard

Sau khi hệ thống chấm điểm xong, trang **Leaderboard** sẽ hiển thị:

- Tên file submission mới nhất.
- Điểm số tương ứng (Public score).
- Vị trí trên bảng xếp hạng.

Phụ lục

1. **Q&A:** Bạn có thể đặt thêm câu hỏi về nội dung bài đọc trong group Facebook hỏi đáp tại [đây](#). Tất cả câu hỏi sẽ được trả lời trong vòng tối đa 4 giờ.

AIO_QAs-Verified

🔒 Nhóm Riêng tư · 1,4K thành viên



Hình 10: Hình ảnh group facebook AIO Q&A